TP02

Filtrer le trafic réseau avec les ACL Cisco

BTS SIO – option SISR | Administration des systèmes et des réseaux

Niveau	BTS SIO option SISR – travail en binôme
Durée	6 h (cours : 1 h – TP : 5 h) – chaque partie du TP indique sa durée conseillée
Outil	Cisco Packet Tracer (8.x recommandé)
Prérequis	TP01 (réseau local : DHCP, DNS, HTTP, routeurs) adressage IPv4 et masques. routage statique de base
Compétences visées	B2.2 Installer, tester et déployer une solution d'infrastructure réseau. B2.3 Exploiter, dépanner et superviser une solution d'infrastructure réseau. Bloc 3 : cybersécurité (sécurisation d'une infrastructure réseau)
À rendre	Compte-rendu Word (captures, commandes, réponses aux questions) + fichier .pkt final + sortie de `show running-config` de R1 et R2

CONSIGNE

Comme pour le TP01 : **capturez chaque étape dans votre compte-rendu Word** (commande saisie + résultat obtenu).

Ce TP est une excellente source pour alimenter votre **portfolio** (thème : sécurisation d'une infrastructure réseau).

Sommaire

Chapitre	Contenu
Chapitre 1 – Cours	1 : pourquoi filtrer 2 : qu'est-ce qu'une ACL 3 : comment le routeur la lit 4 : types d'ACL 5 : wildcard mask 6 : syntaxe 7 : placement et sens 8 : créer et modifier 9 : méthode de conception 10 : limites et bonnes pratiques 11 : questions flash
Annexe	Aide-mémoire ACL (à garder sous les yeux pendant le TP)

Chapitre 1 – Cours : les Access Control Lists (ACL)

OBJECTIF
Ce chapitre vous donne **tout ce qu'il faut savoir** pour réussir le TP : le besoin, le fonctionnement, la syntaxe, le placement et la méthode. Lisez-le **avant** la partie A. Comptez environ 1 h (lecture + questions flash finales).

1 Pourquoi filtrer le trafic ?

Par défaut, un routeur est **poli** : dès qu'il connaît une route vers la destination, il transmet le paquet, sans se demander qui l'envoie ni pourquoi. C'est le comportement voulu pour un réseau qui marche… mais c'est aussi une faille de sécurité. Sans filtrage :

- un poste infecté du service Marketing peut **scanner** et attaquer les serveurs de l'entreprise ;
- un visiteur branché sur le Wi-Fi invité peut **atteindre** les postes de la comptabilité ;
- n'importe quel utilisateur peut tenter de **se connecter en administrateur** sur un routeur.

Il faut donc pouvoir décider **qui a le droit de parler à qui, et pour quel service**. Sur les équipements Cisco, l'outil de base pour cela est l'**ACL** (*Access Control List*, liste de contrôle d'accès).

Utilité	Exemple concret
Cloisonner les réseaux	Le réseau des invités ne doit pas atteindre le réseau des serveurs.
Protéger des serveurs	Seuls HTTP et HTTPS sont ouverts vers le serveur web ; tout le reste est fermé.
Réduire la surface d'attaque	Interdire Telnet (port 23, non chiffré) ou le partage de fichiers SMB vers l'extérieur.
Protéger l'administration	Seul le poste de l'administrateur peut ouvrir une session Telnet/SSH sur le routeur.
Faire respecter une politique	Le service Compta n'accède qu'aux serveurs dont il a besoin.
Limiter des flux inutiles	Empêcher les *pings* (ICMP) depuis une zone non fiable pour compliquer la cartographie du réseau.
Désigner du trafic pour une autre fonction	La même syntaxe sert à choisir le trafic à traduire (NAT), à chiffrer (VPN) ou à prioriser (QoS).

PLACE DES ACL DANS LA SÉCURITÉ
Une ACL est un filtre **simple, gratuit et présent sur tous les routeurs** et switchs de niveau 3. Elle travaille aux couches **3 (IP)** et **4 (TCP/UDP)**. Elle ne remplace pas un **pare-feu** (pas de suivi des connexions, pas d'analyse du contenu) mais elle en est la première brique, dans une logique de **défense en profondeur** (ACL + VLAN + pare-feu + authentification…).

2 Qu'est-ce qu'une ACL ?

Une ACL est une **liste ordonnée de règles**. Chaque règle (appelée **ACE** – *Access Control Entry*) associe :

- une **action** : `permit` (autoriser) ou `deny` (refuser) ;
- des **critères** de correspondance : adresse source, adresse destination, protocole, ports…

Une ACL **n'a aucun effet tant qu'elle n'est pas appliquée** : il faut la lier à une interface (ou à des lignes VTY) et préciser un sens (`in` ou `out`).

L'IMAGE DU VIGILE

Imaginez un vigile devant une salle, avec une liste. Il lit la liste **de haut en bas**. Dès qu'un visiteur correspond à une ligne, il applique la consigne de cette ligne (« entre » ou « dehors ») et ne lit pas la suite. Si le visiteur ne figure sur **aucune** ligne, la consigne finale est : **refusé**.

Anatomie d'une règle

```
access-list 101 deny tcp 192.168.1.0 0.0.0.255 host 172.16.1.100 eq 21
```

Morceau	Rôle	Signification ici
`access-list`	Commande	Ajoute une règle à une ACL numérotée
`101`	Identifiant de l'ACL	ACL étendue (100 à 199)
`deny`	Action	Refuser les paquets qui correspondent
`tcp`	Protocole	Seulement le trafic TCP
`192.168.1.0 0.0.0.255`	Source + wildcard	Tout le réseau 192.168.1.0/24
`host 172.16.1.100`	Destination	Uniquement cette machine
`eq 21`	Port de destination	Port 21 (FTP)

Traduction en français : « refuser le TCP venant du réseau 192.168.1.0/24 et allant vers 172.16.1.100 sur le port 21 (FTP) ».

Vocabulaire à connaître

Terme	Définition
ACL	Liste ordonnée de règles de filtrage.
ACE / règle	Une ligne de l'ACL : action + critères.
permit / deny	Autoriser / refuser.
Wildcard mask	Masque « inversé » qui indique quels bits de l'adresse comparer (voir cours 5).
Inbound (in) / outbound (out)	Sens d'application, vu du **routeur** : paquet qui entre / qui sort par l'interface.
Deny implicite	Règle invisible ajoutée à la fin de toute ACL : `deny ip any any`.
Stateless (sans état)	Le routeur juge chaque paquet isolément, sans mémoire de la connexion.
Trafic de transit	Trafic qui traverse le routeur (celui que filtrent les ACL). À distinguer du trafic **généré par** le routeur.

3 Comment le routeur lit une ACL

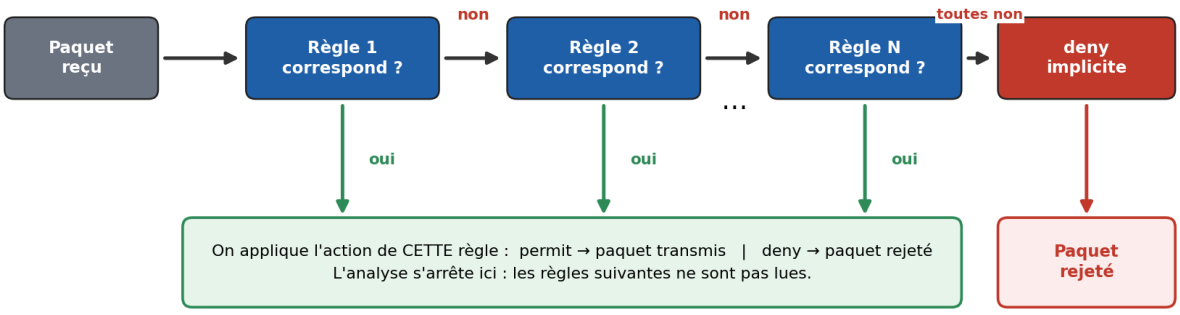


Figure 1 – Algorithme de lecture d'une ACL

Quatre principes à connaître par cœur :

1. **Lecture séquentielle** : le routeur teste les règles dans l'ordre où elles ont été écrites.
2. **Première correspondance** : dès qu'une règle correspond, son action est appliquée et la lecture **s'arrête**. L'ordre des règles est donc **capital**.
3. **Deny implicite** : si aucune règle ne correspond, le paquet est refusé. Une ACL qui ne contient que des `deny` bloque donc **tout** le trafic.
4. **Absence d'état** : chaque paquet est jugé seul. La réponse à une requête autorisée n'est pas automatiquement autorisée.

Exemple guidé : suivre cinq paquets

```
access-list 120 deny tcp 192.168.1.0 0.0.0.255 host 172.16.1.100 eq 21
access-list 120 permit tcp 192.168.1.0 0.0.0.255 any eq 80
access-list 120 permit icmp any any
```

Les trois règles sont numérotées 1, 2 et 3 dans l'ordre d'écriture.

N°	Paquet	Règle qui correspond	Résultat
1	TCP de 192.168.1.111 vers 172.16.1.100, port 21	Règle 1	**Refusé (deny)**
2	TCP de 192.168.1.111 vers 172.16.1.100, port 80	Règle 2 (la règle 1 ne correspond pas : port différent)	**Autorisé**
3	ICMP de 192.168.1.10 vers 172.16.1.10	Règle 3	**Autorisé**
4	TCP de 192.168.1.10 vers 172.16.1.100, port 443	Aucune	**Refusé (deny implicite)**
5	UDP de 192.168.1.10 vers 172.16.1.100, port 53	Aucune	**Refusé (deny implicite)**

> **L'ORDRE FAIT LE RÉSULTAT**
> Si l'on plaçait en première position la règle `permit tcp 192.168.1.0 0.0.0.255 any`, le paquet 1 (FTP vers 172.16.1.100) serait **autorisé** : la règle `deny` ne serait jamais atteinte. Dès qu'une règle générale précède une règle spécifique qui la contredit, la règle spécifique est ignorée. Toujours écrire du **plus précis au plus général**.

Où l'ACL intervient dans le routeur

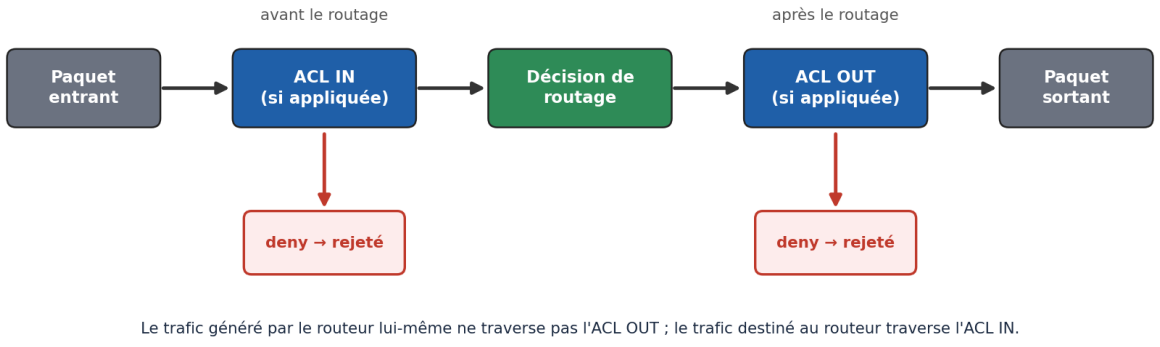


Figure 2 – Trajet d'un paquet et positions des ACL

- Une ACL **`in`** est évaluée **avant** la décision de routage : un paquet refusé n'est même pas routé (économie de ressources).
- Une ACL **`out`** est évaluée **après** la décision de routage, juste avant que le paquet quitte le routeur.
- Une ACL ne filtre **jamais le trafic généré par le routeur** (par exemple un `ping` lancé depuis le routeur), mais une ACL `in` filtre le trafic **destiné** au routeur (ping vers son adresse, Telnet, SSH…).
- Quand un paquet est refusé, le routeur peut renvoyer à l'émetteur un message ICMP « destination unreachable (administratively prohibited) » : c'est ce que vous voyez côté poste.

4 Les types d'ACL

	ACL standard	ACL étendue
Numéros	1 à 99 et 1300 à 1999	100 à 199 et 2000 à 2699
Critère(s)	Adresse **source** uniquement	Protocole, source, destination, ports TCP/UDP, type ICMP, options
Précision	Faible : tout ou rien pour une source	Élevée : on peut viser un service précis
Usage typique	Bloquer/autoriser un poste ou un réseau ; filtrer l'accès aux lignes VTY	Autoriser un service, interdire un protocole, protéger un serveur
Placement	Au plus **près de la destination**	Au plus **près de la source**

Numérotée ou nommée ?

Quel que soit son type, une ACL peut être **numérotée** (identifiée par un numéro) ou **nommée** (identifiée par un nom explicite). Les ACL nommées sont recommandées aujourd'hui.

	ACL numérotée	ACL nommée
Création	`access-list 100 permit …` (une commande par ligne)	`ip access-list extended NOM` puis les règles dans un sous-mode
Lisibilité	Faible (`100`, `101`… : que filtre-t-elle ?)	Bonne (`FILTRE_LAN`, `AGENCE_IN`)
Modification	Ajout en fin de liste seulement ; pour changer l'ordre, il faut tout recréer	Insertion et suppression d'une ligne grâce aux **numéros de séquence**

POUR LA CULTURE (HORS PROGRAMME)
Il existe d'autres familles d'ACL : **réflexives** (autorisent automatiquement le retour d'une connexion initiée de l'intérieur), **dynamiques** (accès temporaire après authentification), **temporelles** (`time-range` : actives à certaines heures) et **IPv6** (toujours nommées). Ce TP se concentre sur les ACL standard et étendues IPv4.

5 Le wildcard mask en détail

Dans une ACL, on ne donne pas un masque de sous-réseau mais un **wildcard mask** (masque générique). Il indique, bit par bit, ce que le routeur doit comparer :

- bit à **0** → le bit de l'adresse **doit être identique** ;
- bit à **1** → le bit est **ignoré** (« peu importe »).

Calcul rapide : wildcard = `255.255.255.255` – masque de sous-réseau. Exemple : pour un /26 (255.255.255.192), le wildcard est 255.255.255.255 – 255.255.255.192 = **0.0.0.63**.

MOYEN MNÉMOTECHNIQUE
Pour un bloc d'adresses aligné, le wildcard de l'octet concerné vaut **(nombre d'adresses du bloc) – 1**. Un /28 contient 16 adresses → wildcard **0.0.0.15**. Un /30 en contient 4 → **0.0.0.3**. Un /25 en contient 128 → **0.0.0.127**.

Exemple détaillé : 192.168.1.32 0.0.0.15

	Dernier octet en binaire	Décimal
Adresse	0010 0000	32
Wildcard	0000 1111	15
Bits comparés (0)	0010 ???? → les 4 premiers bits doivent valoir 0010	–
Plage obtenue	0010 0000 → 0010 1111	**32 à 47**

La règle concerne donc les adresses **192.168.1.32 à 192.168.1.47** (un bloc /28).

Écriture dans l'ACL	Adresses concernées	Équivalent CIDR
`192.168.1.0 0.0.0.255`	192.168.1.0 à 192.168.1.255	/24
`192.168.1.128 0.0.0.127`	192.168.1.128 à 192.168.1.255	/25
`192.168.1.32 0.0.0.15`	192.168.1.32 à 192.168.1.47	/28
`10.1.4.0 0.0.3.255`	10.1.4.0 à 10.1.7.255	/22
`172.16.0.0 0.0.255.255`	172.16.0.0 à 172.16.255.255	/16
`host 192.168.1.10` ≡ `192.168.1.10 0.0.0.0`	Une seule adresse	/32
`any` ≡ `0.0.0.0 255.255.255.255`	Toutes les adresses	/0

ERREUR CLASSIQUE
Écrire le **masque** (`255.255.255.0`) à la place du **wildcard** (`0.0.0.255`). Le routeur l'accepte sans erreur… mais la règle ne correspond plus à ce que vous croyez (voir partie G, cas 3).

6 La syntaxe en détail

6.1 – ACL standard

```
Router(config)# access-list <1-99> {permit | deny} {host <IP> | <réseau> <wildcard> |
any}
```

Exemple	Signification
`access-list 10 permit 192.168.1.0 0.0.0.255`	Autorise le réseau 192.168.1.0/24 (le reste est refusé par le deny implicite).
`access-list 10 deny host 192.168.1.112`	Refuse le poste 192.168.1.112.
`access-list 10 permit any`	Autorise toutes les autres sources.
`access-list 11 permit 192.168.1.10`	Forme abrégée : une adresse sans wildcard équivaut à `host`.

6.2 – ACL étendue

```
Router(config)# access-list <100-199> {permit | deny} <protocole>
                    <source> <wildcard-src>  [opérateur port-src]
                    <destination> <wildcard-dst>  [opérateur port-dst]  [option]
```

Champ	Valeurs	Remarques
Protocole	`ip`, `tcp`, `udp`, `icmp` (et d'autres protocoles IP)	`ip` = tous les protocoles : on ne peut alors pas utiliser de ports.
Source / destination	`host <IP>`, `<réseau> <wildcard>`, `any`	La source vient **toujours avant** la destination.
Opérateur de port	`eq` égal – `neq` différent – `lt` inférieur – `gt` supérieur – `range <min> <max>`	Uniquement avec `tcp` et `udp`.
Option	`established`, `echo`, `echo-reply`, `log`…	Voir ci-dessous.

PORT SOURCE OU PORT DESTINATION ?

La position de l'opérateur de port compte : **juste après la source**, il filtre le **port source** ; **juste après la destination**, il filtre le **port destination**. Un client utilise un port source aléatoire (> 1023) et contacte un port de destination connu : on filtre donc presque toujours le port **destination**.

```
! Requêtes HTTP vers le serveur (port destination 80)
access-list 101 permit tcp any host 172.16.1.100 eq 80
! Réponses du serveur (port source 80)
access-list 102 permit tcp host 172.16.1.100 eq 80 any
```

Ports et protocoles courants

Service	Protocole / port	Service	Protocole / port
FTP (données / contrôle)	TCP 20 / 21	HTTPS	TCP 443
SSH	TCP 22	SMB (partage Windows)	TCP 445
Telnet	TCP 23	RDP (bureau à distance)	TCP 3389

Service	Protocole / port	Service	Protocole / port
SMTP (envoi de mail)	TCP 25	DHCP (serveur / client)	UDP 67 / 68
DNS	UDP 53 (et TCP 53)	TFTP	UDP 69
HTTP (`www`)	TCP 80	SNMP	UDP 161
POP3 / IMAP	TCP 110 / 143	NTP	UDP 123

Cisco accepte aussi des noms à la place des numéros (`eq www`, `eq ftp`, `eq domain`…). Dans `show access-lists`, IOS les réécrit avec ces noms.

Les options à connaître

Option	Utilisation
`established`	TCP uniquement. Correspond aux segments qui portent le drapeau **ACK** ou **RST**, c'est-à-dire les paquets qui **suivent** l'ouverture d'une connexion (réponses). Un premier paquet de connexion (SYN seul) ne correspond pas.
`echo` / `echo-reply`	ICMP. Demande de ping (type 8) / réponse de ping (type 0). Permet de filtrer les demandes sans bloquer les réponses.
`unreachable`, `time-exceeded`	ICMP. « Destination inaccessible » et « TTL dépassé » (utilisé par `traceroute`).
`log`	Journalise chaque correspondance de la règle (utile pour repérer des tentatives d'accès).
`remark <texte>`	Commentaire pour documenter l'ACL (n'est pas une règle).

Exemples commentés

Besoin	Règle
Interdire le HTTP du LAN vers un serveur	`access-list 101 deny tcp 192.168.1.0 0.0.0.255 host 10.0.0.10 eq 80`
Autoriser le HTTPS d'un seul poste vers n'importe où	`access-list 102 permit tcp host 192.168.1.10 any eq 443`
Bloquer les pings vers un serveur	`access-list 103 deny icmp any host 10.0.0.10`
Bloquer les demandes de ping mais pas les réponses	`access-list 103 deny icmp any any echo`
Interdire Telnet partout	`access-list 104 deny tcp any any eq 23`
N'autoriser SSH que depuis le poste d'administration	`access-list 105 permit tcp host 192.168.1.10 any eq 22`
Autoriser FTP (contrôle et données actives)	`access-list 106 permit tcp any host 10.0.0.10 range 20 21`
Refuser tous les ports sauf 80 vers un serveur	`access-list 107 permit tcp any host 10.0.0.10 eq 80` puis `access-list 107 deny ip any host 10.0.0.10`
Refuser tous les ports supérieurs à 1023 vers un serveur	`access-list 108 deny tcp any host 10.0.0.10 gt 1023`

6.3 – Application

```
Router(config)# interface <interface>
Router(config-if)# ip access-group <numéro | nom> {in | out}

! Cas particulier : accès distants au routeur
Router(config)# line vty 0 4
Router(config-line)# access-class <numéro> in
```

Pour les **interfaces**, on utilise `ip access-group` ; pour les **lignes VTY** (Telnet/SSH vers le routeur), on utilise `access-class`.

7 Où et dans quel sens appliquer une ACL ?

Le sens (in / out) se juge du point de vue du routeur

`in` : le paquet **entre** dans le routeur par cette interface. **`out`** : le paquet **sort** du routeur par cette interface. Pour une même conversation, le paquet aller et le paquet retour ne traversent donc pas les interfaces dans le même sens : ce qui est `in` à l'aller devient `out` au retour sur la même interface.

Règles de placement

	Où ?	Pourquoi ?
ACL standard	Au plus **près de la destination**	Elle ne voit que la source. Placée près de la source, elle bloquerait cette source vers **toutes** les destinations.
ACL étendue	Au plus **près de la source**	Elle est précise : on stoppe le trafic indésirable le plus tôt possible, sans le faire traverser le réseau pour rien.

Exemple 1 – ACL standard : interdire un réseau vers un autre

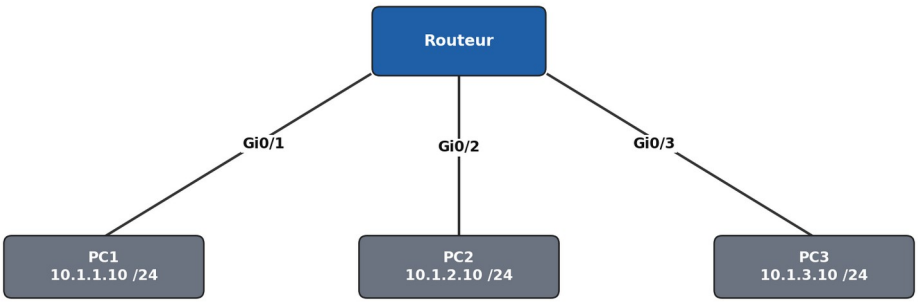


Figure 3 – Trois réseaux reliés à un même routeur

Besoin : le réseau 10.1.1.0/24 (PC1) ne doit pas communiquer avec 10.1.2.0/24 (PC2), mais doit continuer à joindre 10.1.3.0/24 (PC3).

Choix : ACL **standard** (seule la source compte), placée au plus près de la destination, donc sur l'interface **Gi0/2**, dans le sens **out** (le trafic sort vers PC2).

```
Router(config)# access-list 1 deny 10.1.1.0 0.0.0.255
Router(config)# access-list 1 permit any
Router(config)# interface gigabitEthernet 0/2
Router(config-if)# ip access-group 1 out
```

La ligne `permit any` est indispensable : sinon le `deny` implicite bloquerait aussi PC3 vers PC2. Si l'on avait placé cette ACL sur Gi0/1 en `in`, PC1 aurait été bloqué vers **tous** les réseaux, PC3 compris.

Exemple 2 – ACL étendue : interdire le ping d'un réseau vers un autre

Besoin : PC3 ne doit plus pouvoir **pinguer** PC2, PC1 le peut toujours. **Choix** : ACL **étendue** (protocole ICMP), au plus près de la source, donc **Gi0/3** en sens **in**.

```
Router(config)# access-list 100 deny icmp 10.1.3.0 0.0.0.255 10.1.2.0 0.0.0.255
Router(config)# access-list 100 permit ip any any
Router(config)# interface gigabitEthernet 0/3
Router(config-if)# ip access-group 100 in
```

> **UNE SEULE ACL PAR INTERFACE, PAR SENS, PAR PROTOCOLE**
> Appliquer une seconde ACL dans le même sens sur la même interface **remplace** la première. Si vous avez plusieurs besoins sur un même sens, il faut **fusionner** toutes les règles dans une seule ACL (c'est l'objet de la partie E).

8 Créer, lire et modifier une ACL

ACL numérotée : ajout seulement en fin de liste

Chaque commande `access-list <n°> …` ajoute la règle **à la fin**. On ne peut pas en insérer une au milieu ni en supprimer une seule : pour corriger un ordre, on supprime toute l'ACL (`no access-list <n°>`) et on la recrée. Astuce : préparez vos règles dans un éditeur de texte puis collez-les.

ACL nommée : numéros de séquence

```
Router(config)# ip access-list extended FILTRE_LAN
Router(config-ext-nacl)# deny icmp 192.168.1.0 0.0.0.255 1.1.1.0 0.0.0.3
Router(config-ext-nacl)# permit ip any any

Router# show access-lists FILTRE_LAN
Extended IP access list FILTRE_LAN
    10 deny icmp 192.168.1.0 0.0.0.255 1.1.1.0 0.0.0.3 (4 match(es))
    20 permit ip any any (37 match(es))
```

IOS numérote les règles de 10 en 10. Ces numéros servent à **repérer**, **insérer** et **supprimer** des lignes ; le nombre entre parenthèses est le **compteur de correspondances**, précieux pour vérifier qu'une règle est bien utilisée.

Je veux…	Commande (dans le mode de l'ACL nommée)
Ajouter une règle **à la fin**	`permit …` (sans numéro)
Insérer une règle entre 10 et 20	`15 deny …`
Supprimer une ligne	`no 15`
Commenter l'ACL	`remark Bloque le ping vers l'interconnexion`
Supprimer toute l'ACL	`no ip access-list extended NOM` (mode global)
Retirer l'ACL d'une interface sans la supprimer	`no ip access-group NOM in` (mode interface)

> **MODIFIER UNE ACL EN PRODUCTION**
> Supprimer puis recréer une ACL **ligne par ligne** alors qu'elle est appliquée peut couper brutalement le trafic : après la première ligne `deny`, le `deny` implicite bloque tout le reste. Bonne pratique : **construire la nouvelle ACL sous un autre nom**, puis basculer l'interface d'un coup avec `ip access-group NOUVEAU_NOM in`.

9 Méthode pour concevoir une ACL

1. **Formuler le besoin en français** : qui (source) peut faire quoi (protocole/port) vers quoi (destination) ?
2. **Repérer les flux** dans les deux sens (aller **et retour**) et les flux annexes indispensables (DNS, DHCP…).
3. **Choisir le type, le routeur, l'interface et le sens** (règles de placement).
4. **Écrire les règles** du plus précis au plus général, en décidant du sort du trafic restant (`permit ip any any` ou `deny` implicite).
5. **Appliquer, tester (cas autorisés ET cas interdits), vérifier les compteurs, documenter.**

Exemple complet

Besoin : dans une PME, le réseau **Compta** (192.168.10.0/24) doit pouvoir joindre uniquement : le serveur de fichiers 192.168.50.10 (SMB, TCP 445), le serveur web interne 192.168.50.20 (HTTP et HTTPS) et le serveur DNS 192.168.50.53 (UDP 53). Le reste du réseau Serveurs (192.168.50.0/24) lui est interdit. Tout autre trafic de la Compta (Internet…) reste autorisé.

- **Flux** : Compta → serveurs (requêtes). Le retour est géré par le routeur des serveurs, hors ACL ici. On n'oublie pas le **DNS**, sinon la résolution de noms casse.
- **Type** : étendue (ports). **Placement** : près de la source → interface Compta du routeur (`Gi0/1`), sens **in**.
- **Ordre** : autorisations précises d'abord, interdiction générale du réseau Serveurs ensuite, `permit ip any any` à la fin.

```
ip access-list extended COMPTA_IN
 remark Compta : acces limite aux serveurs necessaires
 permit tcp 192.168.10.0 0.0.0.255 host 192.168.50.10 eq 445
 permit tcp 192.168.10.0 0.0.0.255 host 192.168.50.20 eq 80
 permit tcp 192.168.10.0 0.0.0.255 host 192.168.50.20 eq 443
 permit udp 192.168.10.0 0.0.0.255 host 192.168.50.53 eq 53
 deny ip 192.168.10.0 0.0.0.255 192.168.50.0 0.0.0.255
 permit ip any any
!
interface gigabitEthernet 0/1
 ip access-group COMPTA_IN in
```

Tests à prévoir : cas autorisés (accès SMB, page web, résolution DNS) **et** cas interdits (Telnet vers 192.168.50.10, ping vers 192.168.50.20, accès à un autre serveur du réseau 192.168.50.0/24), puis `show access-lists COMPTA_IN` pour contrôler les compteurs.

Cours 10 Limites et bonnes pratiques

Limites

- **Sans état** : la réponse à une requête autorisée doit être autorisée explicitement (mot-clé `established` pour TCP, `echo-reply` pour ICMP). Pour un vrai suivi des connexions, il faut un **pare-feu à états**.

- **Filtrage par adresse IP** : une adresse se change ou s'usurpe ; en DHCP elle varie. On préfère filtrer par **réseau** (VLAN, sous-réseau de rôle) que par poste.
- **Pas d'analyse applicative** : une ACL ne sait pas distinguer un site légitime d'un site malveillant sur le port 443.
- **Complexité** : plus une ACL grossit, plus elle est difficile à maintenir et à auditer.

Bonnes pratiques

- **Documenter** chaque ACL (`remark`, nom explicite) et conserver la politique de sécurité qui la justifie.
- **Précis d'abord, général ensuite** ; ne mettre `permit ip any any` en fin d'ACL que si l'on raisonne en « liste noire ». En zone sensible, préférer une **liste blanche** (tout refuser sauf le nécessaire).
- **Tester avant de généraliser** : cas autorisés ET interdits, puis test de non-régression après chaque modification.
- **Ne pas se couper l'accès** : lors du filtrage de Telnet/SSH, garder un accès par la console.
- **Sauvegarder** la configuration (`copy running-config startup-config`) et l'ACL avant modification.
- **Combiner** les protections : ACL + segmentation VLAN + pare-feu + authentification.

Cours 11 Questions flash (auto-évaluation)

Répondez sans regarder le cours ; le corrigé est fourni à votre enseignant.

N°	Question	Réponses proposées
1	Une ACL ne contient que la ligne `deny icmp any any` et est appliquée sur une interface. Que devient le trafic TCP ?	a) Il passe b) Il est bloqué c) Cela dépend du port
2	Quel est le wildcard du réseau 192.168.10.64/26 ?	a) 0.0.0.63 b) 0.0.0.64 c) 255.255.255.192
3	Sur quoi une ACL standard filtre-t-elle ?	a) La source b) La source et la destination c) La destination et le port
4	Où place-t-on une ACL étendue ?	a) Près de la destination b) Près de la source c) Sur n'importe quel routeur
5	Plusieurs règles correspondent à un paquet. Laquelle s'applique ?	a) La plus précise b) La première dans la liste c) Toutes
6	Un `ping` est lancé depuis le routeur R1. Une ACL `out` de R1 refuse l'ICMP. Le ping est-il bloqué ?	a) Oui b) Non, le trafic généré par le routeur n'est pas filtré c) Seulement la réponse
7	Dans `permit tcp any host 10.0.0.10 eq 443`, que filtre `eq 443` ?	a) Le port source b) Le port destination c) Les deux
8	Combien d'ACL IPv4 entrantes peut-on appliquer sur une interface ?	a) 1 b) 2 c) Autant qu'on veut
9	Quelle commande affiche les ACL et leurs compteurs de correspondances ?	a) `show ip route` b) `show access-lists` c) `show ip interface brief`
10	Quel mot-clé reconnaît les réponses d'une connexion TCP initiée de l'intérieur ?	a) `established` b) `log` c) `remark`

1 b · 2 a · 3 a · 4 b · 5 b · 6 b · 7 b · 8 a · 9 b · 10 a

Travaux pratiques

1. Contexte et objectifs

Vous êtes technicien(ne) réseau dans l'entreprise fictive **SISR2** (domaine `sisr2.local`). Le **siège** (LAN 192.168.1.0/24 : serveurs DHCP, DNS et Web) est relié par une liaison série 1.1.1.0/30 à une **agence** (LAN 172.16.1.0/24). Suite à un audit, le RSSI vous transmet la politique de filtrage suivante. Votre mission : **la traduire en ACL Cisco**, la tester, puis la documenter.

Règle	Énoncé de la politique de sécurité
R1	Les postes du LAN du siège ne doivent pas pouvoir envoyer de **ping** vers le réseau d'interconnexion **1.1.1.0/30**.
R2	**PC2** (192.168.1.112) ne doit avoir **aucun accès** au réseau de l'agence.
R3	Depuis le LAN du siège, le **Serveur-Agence** (172.16.1.100) n'est accessible qu'en **HTTP**. Le reste de l'agence (ex. PC3) reste joignable normalement.
R4	Depuis l'agence, seuls sont autorisés vers le siège : le **serveur Web** (HTTP) et le **serveur DNS** (DNS). Les **réponses** aux connexions initiées depuis le siège doivent continuer à passer.

Objectifs pédagogiques

À l'issue du TP, vous serez capable de :

- expliquer le rôle d'une ACL et **l'ordre dans lequel le routeur la lit** (règles évaluées de haut en bas, `deny` implicite final) ;
- **calculer un wildcard mask** et lire une règle d'ACL ;
- choisir le **type** (standard / étendue), le **routeur**, l'**interface** et le **sens** (`in` / `out`) adaptés à un besoin ;
- écrire, appliquer, tester et vérifier une ACL **numérotée** puis **nommée**, et la modifier sans tout recréer ;
- **diagnostiquer et corriger** une ACL défaillante ;
- identifier les **limites** des ACL (filtrage sans état, adresses IP dynamiques).

2. Topologie et plan d'adressage.

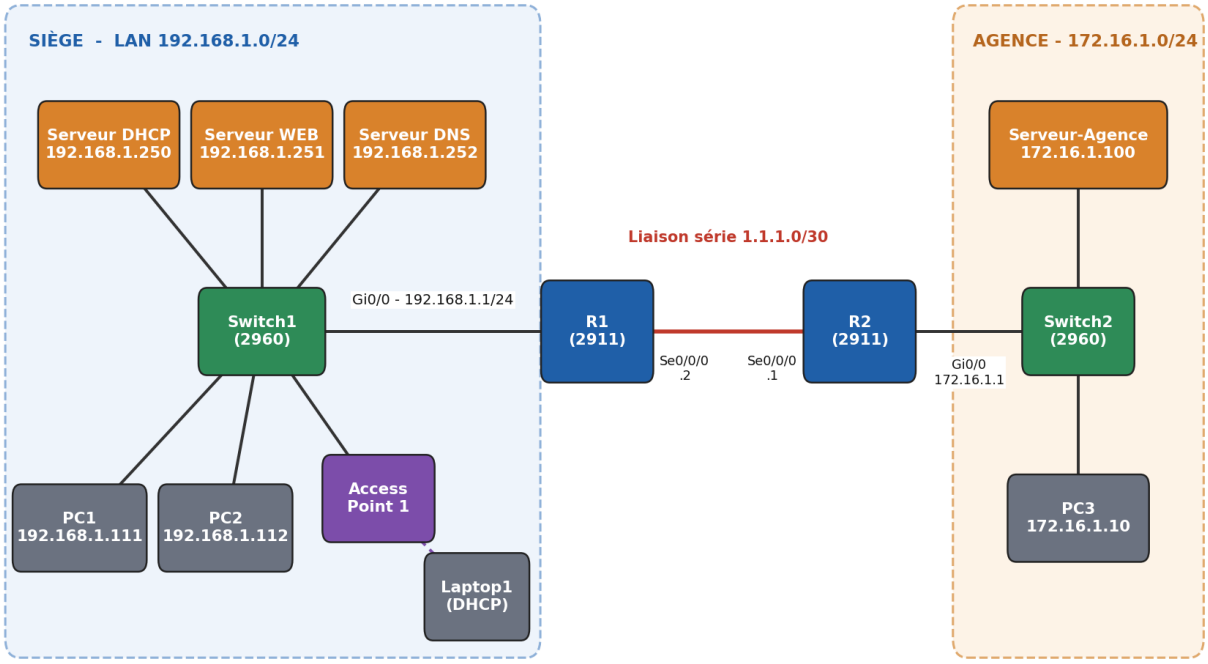


Figure 4 – Topologie du TP02

Équipement	Interface	Adresse IP / masque	Passerelle	Remarque
R1	Gi0/0	192.168.1.1 /24	–	Vers Switch1 (LAN siège)
R1	Se0/0/0	1.1.1.2 /30	–	Vers R2
R2	Se0/0/0	1.1.1.1 /30	–	Vers R1
R2	Gi0/0	172.16.1.1 /24	–	Vers Switch2 (agence)
Serveur DHCP	Fa0	192.168.1.250 /24	192.168.1.1	Pool 192.168.1.10 → .110
Serveur WEB	Fa0	192.168.1.251 /24	192.168.1.1	www.sisr2.local
Serveur DNS	Fa0	192.168.1.252 /24	192.168.1.1	www.sisr2.local → .251
PC1	Fa0	192.168.1.111 /24	192.168.1.1	**IP statique** (hors pool), DNS .252
PC2	Fa0	192.168.1.112 /24	192.168.1.1	**IP statique** (hors pool), DNS .252
Laptop1	Wi-Fi	DHCP	192.168.1.1	Via Access Point 1
PC3	Fa0	172.16.1.10 /24	172.16.1.1	DNS 192.168.1.252
Serveur-Agence	Fa0	172.16.1.100 /24	172.16.1.1	Services HTTP et FTP activés

POURQUOI PC1 ET PC2 EN IP STATIQUE ?

Une ACL filtre sur des **adresses IP**. Si PC2 obtenait son adresse par DHCP, elle pourrait changer et la règle « bloquer PC2 » ne s'appliquerait plus au bon poste. Vous reviendrez sur cette limite en partie C et dans le bilan.

Partie A – Préparer l'environnement (30 min)

A1. Créez la base : Switch1, serveurs, PC1, PC2, Access Point, Laptop1, R1 et R2). **Ajoutez** un **Switch2 (2960)** relié à l'interface Gi0/0 de R2, puis **PC3** et **Serveur-Agence** reliés à Switch2.

> **RAPPEL**
> Un routeur 2911 n'a pas de port série par défaut : **éteignez-le**, ajoutez un module **HWIC-2T**, puis rallumez-le. Utilisez un câble **série DCE** entre R1 et R2. La commande `clock rate` se saisit uniquement sur l'extrémité **DCE** (petite horloge sur le câble) ; ci-dessous on suppose que c'est R1.

A2. Configurez les équipements selon le plan d'adressage. Sur **Serveur-Agence**, activez les services **HTTP** et **FTP** (Services → FTP : utilisateur `cisco`, mot de passe `cisco`). Passez PC1 et PC2 en IP statique (avec DNS 192.168.1.252) ; Laptop1 reste en DHCP.

A3. Configurez les routeurs (extrait à saisir en mode CLI) :

Routeur R1

```
Router> enable
Router# configure terminal
Router(config)# hostname R1
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip address 192.168.1.1 255.255.255.0
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# interface serial 0/0/0
R1(config-if)# ip address 1.1.1.2 255.255.255.252
R1(config-if)# clock rate 64000
R1(config-if)# no shutdown
R1(config-if)# exit
R1(config)# ip route 172.16.1.0 255.255.255.0 1.1.1.1
```

R1 : Configuration, Attention → interface serial 0/3/0 pas interface serial 0/0/0

```
Router>en
Router#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R1
R1(config)#interface gigabitEthernet 0/0
R1(config-if)#ip address 192.168.1.1 255.255.255.0
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up
exit
```

```
R1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#interface serial 0/3/0
R1(config-if)#ip address 1.1.1.2 255.255.255.252
R1(config-if)#clock rate 64000
R1(config-if)#no shutdown

%LINK-5-CHANGED: Interface Serial0/3/0, changed state to down
R1(config-if)#exit
R1(config)#ip route 172.16.1.0 255.255.255.0 1.1.1.1
R1(config)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console
```

Routeur R2

```
Router> enable
Router# configure terminal
Router(config)# hostname R2
R2(config)# interface serial 0/0/0
R2(config-if)# ip address 1.1.1.1 255.255.255.252
R2(config-if)# no shutdown
R2(config-if)# exit
R2(config)# interface gigabitEthernet 0/0
R2(config-if)# ip address 172.16.1.1 255.255.255.0
R2(config-if)# no shutdown
R2(config-if)# exit
R2(config)# ip route 192.168.1.0 255.255.255.0 1.1.1.2
```

R2 : Configuration, Attention → interface serial 0/3/0 pas interface serial 0/0/0

```
Router>en
Router#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R2
R2(config)#interface serial 0/3/0
R2(config-if)#ip address 1.1.1.1 255.255.255.252
R2(config-if)#no shut

R2(config-if)#
%LINK-5-CHANGED: Interface Serial0/3/0, changed state to up
exit
R2(config)#
R2(config)#interface gigabitEthernet 0/0
R2(config-if)#ip address 172.16.1.1 255.255.255.0
R2(config-if)#no shutdown

R2(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0, changed state to up

R2(config-if)#exit
R2(config)#ip route 192.168.1.0 255.255.255.0 1.1.1.2
R2(config)#end
R2#
%SYS-5-CONFIG_I: Configured from console by console
```

R1 : show ip route

```
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     1.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       1.1.1.0/30 is directly connected, Serial0/3/0
L       1.1.1.2/32 is directly connected, Serial0/3/0
     172.16.0.0/24 is subnetted, 1 subnets
S       172.16.1.0/24 [1/0] via 1.1.1.1
     192.168.1.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.1.0/24 is directly connected, GigabitEthernet0/0
L       192.168.1.1/32 is directly connected, GigabitEthernet0/0
```

R1 : show ip interface brief

```
R1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     192.168.1.1     YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  administratively down down
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Serial0/3/0            1.1.1.2         YES manual up                    up
Serial0/3/1            unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
R1#
```

R2 : show ip route

```
R2#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     1.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       1.1.1.0/30 is directly connected, Serial0/3/0
L       1.1.1.1/32 is directly connected, Serial0/3/0
     172.16.0.0/16 is variably subnetted, 2 subnets, 2 masks
C       172.16.1.0/24 is directly connected, GigabitEthernet0/0
L       172.16.1.1/32 is directly connected, GigabitEthernet0/0
S    192.168.1.0/24 [1/0] via 1.1.1.2
```

R2 : show ip interface brief

```
R2#show ip route brief
Translating "brief"...domain server (255.255.255.255)
% Invalid input detected

R2#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0     172.16.1.1      YES manual up                    up
GigabitEthernet0/1     unassigned      YES unset  administratively down down
GigabitEthernet0/2     unassigned      YES unset  administratively down down
Serial0/3/0            1.1.1.1         YES manual up                    up
Serial0/3/1            unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
R2#
```

Les deux commandes `ip route` sont des **routes statiques** : chaque routeur apprend où se trouve le LAN de l'autre côté. Sans elles, les paquets aller arriveraient mais les réponses ne reviendraient jamais.

A4. Vérifiez : `show ip route` sur R1 et R2 (routes `C` et `S` attendues) puis `show ip interface brief` (interfaces **up/up**). Capturez les résultats. **(FAIT AU DESSUS)**

A5. Réalisez les **14 tests de référence** du tableau ci-dessous **avant toute ACL** et remplissez la colonne « Avant ACL » (OK / KO). Enregistrez le fichier sous `TP02_avant_ACL.pkt`.

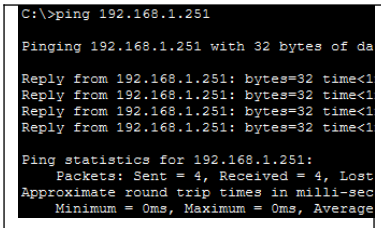
> **ASTUCE PACKET TRACER**
> Ping : Desktop → Command Prompt → `ping <IP>`. Le premier ping peut échouer (résolution ARP) : relancez-le. Web : Desktop → Web Browser. FTP : `ftp 172.16.1.100` dans le Command Prompt. Le mode **Simulation** montre précisément où un paquet est arrêté.

N°	Depuis	Vers	Test	Avant ACL	Prévu après ACL	Obtenu après ACL
1	PC1	Serveur Web (192.168.1.251)	Ping	OK	OK	OK
2	PC1	R1 – Se0/0/0 (1.1.1.2)	Ping	OK	KO	KO
3	PC1	R2 – Se0/0/0 (1.1.1.1)	Ping	OK	KO	KO
4	PC1	PC3 (172.16.1.10)	Ping	OK	OK	OK
5	PC2	PC3 (172.16.1.10)	Ping	OK	KO	KO
6	PC1	Serveur-Agence (172.16.1.100)	Ping	OK	KO	KO
7	PC1	Serveur-Agence	HTTP	OK	OK	OK
8	PC1	Serveur-Agence	FTP	OK	KO	KO
9	Laptop1	Serveur-Agence	HTTP	OK	OK	OK
10	PC2	Serveur-Agence	HTTP	OK	KO	KO
11	PC3	www.sisr2.local	DNS + HTTP	OK	OK	OK
12	PC3	Serveur DNS (192.168.1.252)	Ping	OK	KO	KO
13	PC3	PC1 (192.168.1.111)	Ping	OK	KO	KO
14	R2	PC1 (192.168.1.111)	Ping	OK	KO	KO

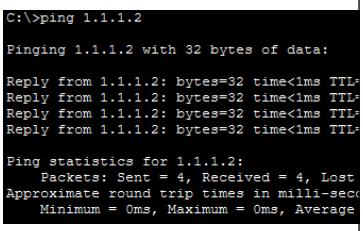
Avant ACL, les 14 tests sont OK (voir captures ci-dessous). Colonne « Prévu » : situation finale attendue avec les ACL des parties C (ACL 10), E (FILTRE_LAN) et F (AGENCE_IN). Colonne « Obtenu » : résultats obtenus après mise en place des ACL.

AVANT ACL SCREENS

N1

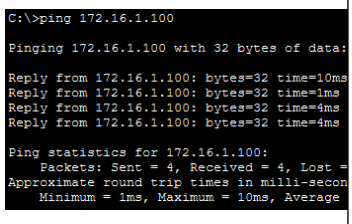


N2



N3

N6



N7

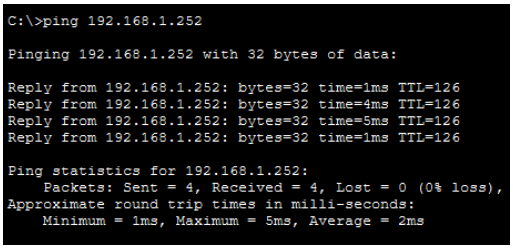


N8

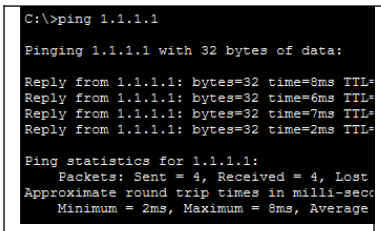
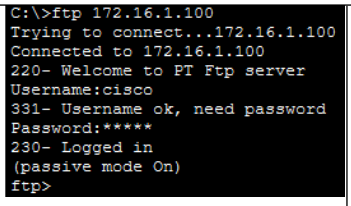
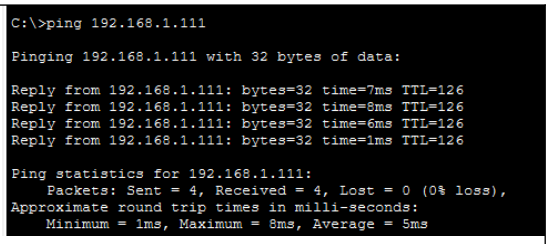
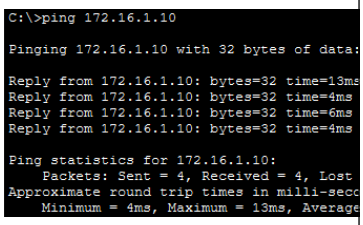
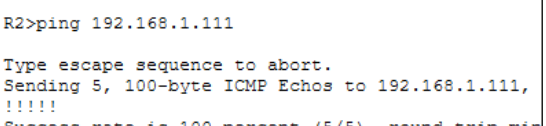
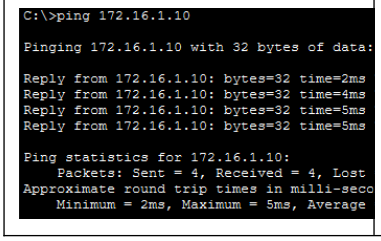
N11



N12



N13

<table>
<tr>
<td> 
</td>
<td> 
</td>
<td> 
</td>
</tr>
<tr>
<td>N4
 
</td>
<td>N9
Capture non disponible</td>
<td>N14
 
</td>
</tr>
<tr>
<td>N5
 
</td>
<td>N10
Capture non disponible</td>
<td></td>
</tr>
</table>

Partie B – Comprendre avant de configurer (30 min)

Relisez au besoin le **Chapitre 1 (cours)** et l'**Annexe – Aide-mémoire ACL**, puis répondez **sans utiliser le simulateur**.

B1 – Wildcard masks

Un **wildcard mask** est l'inverse d'un masque de sous-réseau : un bit à `0` = « doit être identique », un bit à `1` = « peu importe ». Complétez :

Réseau / masque	Wildcard à écrire dans l'ACL
192.168.1.0 /24	0.0.0.255
1.1.1.0 /30	0.0.0.3
192.168.1.128 /25	0.0.0.127
10.0.0.0 /8	0.255.255.255
Hôte unique 172.16.1.100	0.0.0.0

Tableau correct : les 5 wildcards sont justes.

Écriture dans une ACL	Plage d'adresses concernée
`192.168.1.0 0.0.0.15`	192.168.1.0 à 192.168.1.15 (/28, 16 adresses)
`192.168.1.64 0.0.0.63`	192.168.1.64 à 192.168.1.127 (/26, 64 adresses)
`172.16.0.0 0.15.255.255`	172.16.0.0 à 172.31.255.255 (/12)
`any`	Toutes les adresses : 0.0.0.0 à 255.255.255.255 (/0)

B2 – Ordre de lecture et deny implicite

```
R1(config)# access-list 110 permit ip any any
R1(config)# access-list 110 deny icmp 192.168.1.0 0.0.0.255 any
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip access-group 110 in
```

B2.1 Un ping de PC1 vers PC3 est-il bloqué ? Justifiez. Que faudrait-il changer pour que le ping soit bloqué ?

Non, le ping n'est pas bloqué. Le routeur lit les règles dans l'ordre : la première, `permit ip any any`, correspond à tous les paquets IP (ICMP compris). Le paquet est donc autorisé et la règle `deny icmp` n'est jamais lue.

Pour bloquer le ping, il faut placer `deny icmp 192.168.1.0 0.0.0.255 any` **avant** `permit ip any any`. Avec une ACL numérotée, on doit supprimer l'ACL 110 (`no access-list 110`) puis la recréer dans le bon ordre.

B2.2 Si l'on supprime la ligne `permit ip any any` et qu'il ne reste que la règle `deny icmp…`, que devient le trafic **HTTP** du LAN vers l'agence ? Pourquoi ?

Le trafic HTTP du LAN vers l'agence est bloqué. La seule règle restante (`deny icmp …`) ne correspond pas à un paquet TCP, donc aucune règle ne correspond et le **deny implicite** (`deny ip any any`) s'applique. En réalité, tout le trafic du LAN vers l'extérieur est alors bloqué, pas seulement l'ICMP.

B3 – Sens in / out

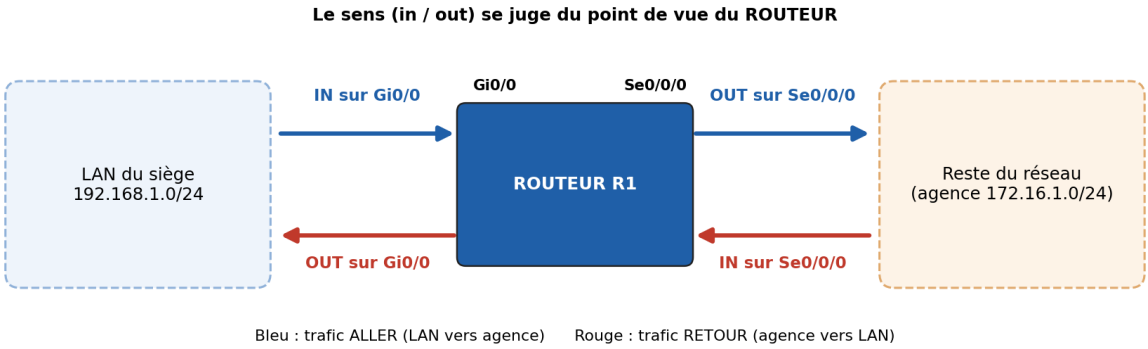


Figure 5 – Le sens d'une ACL se juge du point de vue du routeur

B3.1 Un ping part de PC1 vers PC3. Complétez le tableau (in / out) pour chaque interface traversée, à l'aller puis au retour :

Interface	Paquet aller (PC1 → PC3)	Paquet retour (PC3 → PC1)
R1 – Gi0/0	in	out
R1 – Se0/0/0	out	in
R2 – Se0/0/0	in	out
R2 – Gi0/0	out	in

B4 – Lire une règle

B4.1 Expliquez en une phrase chacune des règles suivantes :

Règle	Explication en français
`access-list 110 permit tcp 192.168.1.0 0.0.0.255 host 172.16.1.100 eq 21`	Autorise le TCP du réseau 192.168.1.0/24 vers le serveur 172.16.1.100 sur le port 21 (FTP, connexion de contrôle).
`access-list 111 deny tcp host 192.168.1.111 host 172.16.1.100 eq www`	Refuse le HTTP (TCP 80, « www ») de PC1 (192.168.1.111) vers le Serveur-Agence 172.16.1.100.
`access-list 111 permit ip any any`	Autorise tout le reste du trafic IP, quelles que soient la source et la destination.
`access-list 112 deny udp any host 192.168.1.252 eq 53`	Refuse les requêtes DNS (UDP 53) de toute source vers le serveur DNS 192.168.1.252 (sans autre règle, le deny implicite bloque aussi tout le reste).

B5 – Standard ou étendue ?

B5.1 Pour chaque besoin, indiquez le type d'ACL nécessaire (standard ou étendue) et justifiez :

Besoin	Type	Justification
Interdire à PC2 toute communication vers l'agence	Standard	Seule la source (PC2) compte : on bloque tout son trafic vers l'agence.
Interdire uniquement le FTP de PC2 vers le Serveur-Agence	Étendue	Il faut préciser le protocole et le port (TCP 21) ainsi que la destination (Serveur-Agence).
N'autoriser que le HTTP de l'agence vers le serveur Web du siège	Étendue	Il faut filtrer sur le protocole (TCP), le port (80) et la destination précise (serveur Web).

Partie C – ACL standard : appliquer la règle R2 (35 min)

> **OBJECTIF**
> Bloquer **PC2** (192.168.1.112) vers l'agence tout en laissant PC1 et Laptop1 y accéder. Une ACL standard ne regarde que l'**adresse source**.

C1. Quel routeur, quelle interface, quel sens ? Justifiez avec la règle de placement des ACL standard (voir annexe).

Routeur **R2**, interface **Gi0/0**, sens **out**. Une ACL standard ne voit que l'adresse source : on la place au plus près de la destination (le LAN de l'agence). Le trafic de PC2 sort de R2 par Gi0/0 vers l'agence, d'où le sens out. Sur R1, elle couperait PC2 de toutes les destinations.

C2. Complétez puis saisissez les commandes (remplacez les pointillés) :

```
R?(config)# access-list 10 …… host ……
R?(config)# access-list 10 …… any
R?(config)# interface ……
R?(config-if)# ip access-group 10 ……
```

```
R2(config)# access-list 10 deny host 192.168.1.112
R2(config)# access-list 10 permit any
R2(config)# interface gigabitEthernet 0/0
```

```
R2(config-if)# ip access-group 10 out
```

C3. Vérifiez avec `show access-lists` et `show ip interface gigabitEthernet 0/0` (repérez la ligne « access list is »). Capturez.

`show access-lists` affiche « Standard IP access list 10 » avec ses deux règles. `show ip interface gigabitEthernet 0/0` sur R2 doit indiquer « Outgoing access list is 10 ».

C4. Lancez les tests **4, 5 et 10**, puis relancez `show access-lists` : que remarquez-vous à côté de chaque règle ? Reportez vos résultats dans le tableau de la partie A.

Test 4 (PC1 → PC3) : OK. Test 5 (PC2 → PC3) : KO. Test 10 (PC2 → HTTP Serveur-Agence) : KO. Dans `show access-lists`, chaque règle affiche désormais un compteur « (n match(es)) » : `deny host 192.168.1.112` augmente avec les tests 5 et 10, `permit any` avec le test 4. Les compteurs prouvent que l'ACL est bien utilisée.

C5. Que se passerait-il si la **même ACL** était placée en **entrée de Gi0/0 sur R1** ? Recréez-la sur R1 (`access-list 10 …`), appliquez-la, puis testez : PC2 → PC3, PC2 → 192.168.1.1 (passerelle) et PC2 → Serveur Web. Annulez ensuite (`no ip access-group 10 in` puis `no access-list 10`).

En entrée de Gi0/0 sur R1, l'ACL bloque PC2 dès son entrée dans le routeur : PC2 → PC3 KO, et PC2 → 192.168.1.1 (passerelle) KO, car une ACL in filtre aussi le trafic destiné au routeur. En revanche, PC2 → Serveur Web reste OK : ce trafic reste dans le LAN et ne passe pas par le routeur.

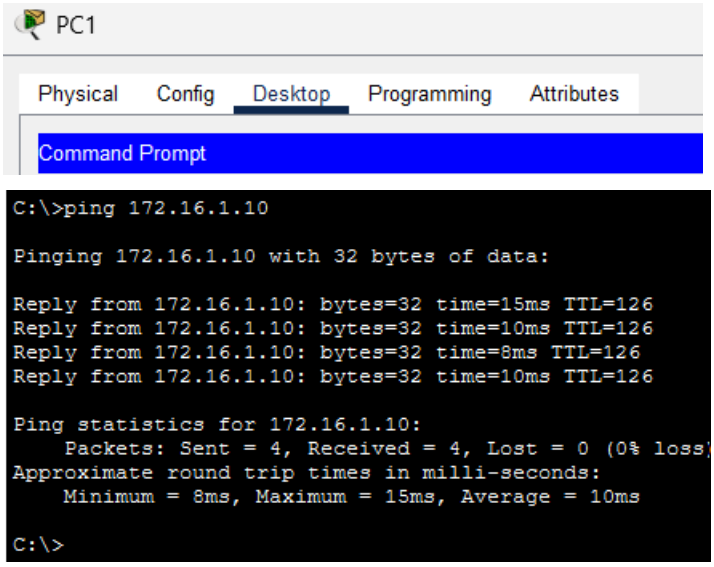
PC2 serait donc coupé de **tous** les réseaux distants (agence, interconnexion…) et pas seulement de l'agence. C'est pour cela qu'une ACL standard se place près de la destination.

C6. Changez l'adresse IP de PC2 en 192.168.1.113 et refaites le test 5. Que constatez-vous ? Que pouvez-vous en conclure sur le filtrage par adresse IP ? Remettez ensuite l'adresse .112.

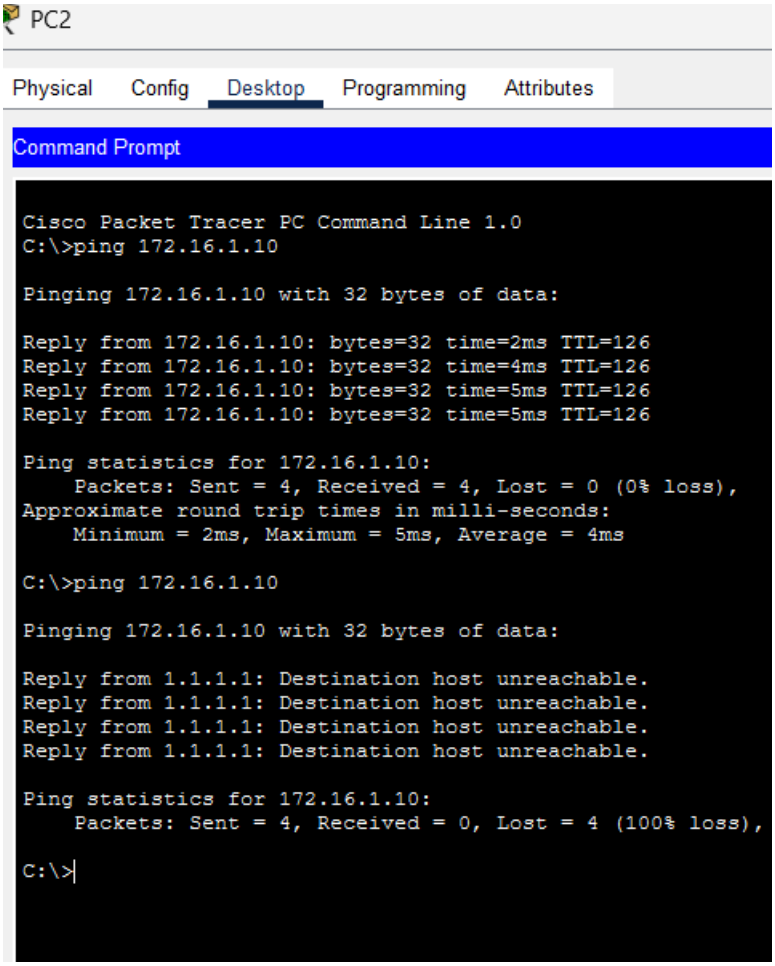
Avec l'adresse 192.168.1.113, le test 5 redevient OK : l'ACL bloque une adresse, pas un poste. Un utilisateur qui change son IP (ou une adresse qui change via DHCP) contourne le filtrage. Le filtrage par adresse IP d'un poste est donc fragile ; il faut le compléter : réservation DHCP, port-security, 802.1X, ou un VLAN dédié filtré par sous-réseau.

> **SAUVEGARDE**
> Tapez `copy running-config startup-config` sur R2 et enregistrez `TP02_partieC.pkt`. Si une partie ultérieure se passe mal, vous pourrez revenir en arrière.

TEST PC1 ping 172.16.1.10



TEST PC2 ping 172.16.1.10



Partie D – ACL étendue numérotée : appliquer la règle R1 (40 min)

> **OBJECTIF**
> Bloquer les **pings** du LAN du siège vers le réseau d'interconnexion **1.1.1.0/30**, sans gêner les autres flux. Une ACL étendue filtre sur la source, la destination, le protocole et les ports.

D1. Pourquoi une ACL **étendue** est-elle nécessaire ici ? Sur quel routeur, quelle interface et quel sens la placer ? (règle de placement des ACL étendues)

Il faut filtrer sur le **protocole** (ICMP) et sur la **destination** (1.1.1.0/30) : une ACL standard ne voit que la source et bloquerait tout le trafic du LAN. Une ACL étendue se place au plus près de la source : routeur **R1**, interface **Gi0/0**, sens **in**.

D2. Complétez puis saisissez :

```
R1(config)# access-list 100 …… icmp …… …… …… ……
R1(config)# access-list 100 …… ip any any
R1(config)# interface ……
R1(config-if)# ip access-group 100 ……
```

```
R1(config)# access-list 100 deny icmp 192.168.1.0 0.0.0.255 1.1.1.0 0.0.0.3
R1(config)# access-list 100 permit ip any any
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip access-group 100 in
```

Conforme à ma capture de configuration ci-dessous.

D3. Pourquoi faut-il la deuxième ligne (`permit ip any any`) ? Vérifiez votre réponse en la supprimant temporairement (`no access-list 100`, puis recréation de la seule première ligne ; l'ACL reste appliquée sur l'interface) : que devient l'accès au Web vers l'agence ? Rétablissez ensuite l'ACL complète.

Sans `permit ip any any`, le deny implicite refuse tout ce qui n'est pas explicitement autorisé : en plus du ping vers l'interconnexion, tout le trafic du LAN vers l'extérieur est bloqué. L'accès Web à l'agence (test 7) devient KO. Il redevient OK une fois l'ACL complète rétablie.

D4. Effectuez les tests **1, 2, 3 et 4**, puis : (a) `ping 1.1.1.1` **depuis R1** ; (b) `ping 192.168.1.111` **depuis R2**. Expliquez chaque résultat. En particulier : pourquoi (a) fonctionne-t-il alors que la règle vise 1.1.1.0/30 ? Pourquoi (b) échoue-t-il alors que R2 n'est concernée par aucune ACL ?

Tests : 1 OK (même LAN, pas de passage par R1), 2 KO, 3 KO (pings vers 1.1.1.0/30 refusés), 4 OK (PC3 n'est pas dans 1.1.1.0/30).

(a) `ping 1.1.1.1` depuis R1 : **OK**. Ce trafic est généré par le routeur lui-même : une ACL ne le filtre pas. De plus, il ne traverse pas Gi0/0 en entrée.

(b) `ping 192.168.1.111` depuis R2 : **KO**. La demande (source 1.1.1.1) arrive bien jusqu'à PC1, mais sa réponse (echo-reply de 192.168.1.111 vers 1.1.1.1) entre dans R1 par Gi0/0 et correspond à `deny icmp 192.168.1.0 0.0.0.255 1.1.1.0 0.0.0.3` : elle est rejetée. L'ACL est sans état : elle ne sait pas que ce paquet est une réponse.

> **À RETENIR**
> Une ACL **ne filtre pas le trafic généré par le routeur lui-même**. Et une ACL Cisco classique est **sans état** : elle ne « se souvient » pas d'une requête, chaque paquet (y compris la réponse) est jugé indépendamment.

D5. Modifiez la règle pour ne bloquer que les **demandes** de ping (mot-clé `echo` en fin de règle). Pour cela, supprimez l'ACL 100 et recréez-la avec le mot-clé. Refaites (b) : que change-t-il ? Quel est l'intérêt de cette version ? **Revenez ensuite à la version initiale (sans `echo`)** pour la suite du TP.

```
R1(config)# no access-list 100
```

```
R1(config)# access-list 100 deny icmp 192.168.1.0 0.0.0.255 1.1.1.0 0.0.0.3 echo
R1(config)# access-list 100 permit ip any any
```

Le test (b) devient OK : seules les **demandes** de ping (echo, type 8) du LAN sont refusées, les **réponses** (echo-reply, type 0) passent. Intérêt : les postes ne peuvent toujours pas pinguer l'interconnexion, mais l'administrateur garde la possibilité de pinguer le LAN depuis les routeurs pour la supervision et le dépannage.

```
R1>en
R1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#access-list 100 deny icmp 192.168.1.0 0.0.0.255 1.1.1.0 0.0.0.3
R1(config)#access-list 100 permit ip any any
R1(config)#interface gigabitEthernet 0/0
R1(config-if)#ip access-group 100 in
R1(config-if)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#
```

TEST PC : Ping de PC1 vers 1.1.1.1 et 1.1.1.2 = KO.

```
C:\>ping 1.1.1.1

Pinging 1.1.1.1 with 32 bytes of data:

Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.

Ping statistics for 1.1.1.1:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

```
C:\>ping 1.1.1.2

Pinging 1.1.1.2 with 32 bytes of data:

Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.

Ping statistics for 1.1.1.2:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

TEST Ping de R2 vers 192.168.1.111

```
R2>ping 192.168.1.111

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.1.111, timeout is 2 seconds:
.....
Success rate is 0 percent (0/5)
```

Partie E – ACL nommées, ordre et fusion : appliquer la règle R3 (50 min)

> **OBJECTIF**
> Depuis le LAN, le **Serveur-Agence** ne doit être accessible qu'en **HTTP**. Le problème : R1 possède déjà l'ACL 100 en entrée sur Gi0/0, et **une interface n'accepte qu'une ACL par sens**. Il faut donc **fusionner** les règles R1 et R3 dans une seule ACL, **nommée**.

Une ACL nommée se modifie ligne par ligne grâce aux **numéros de séquence** (10, 20, 30…), ce que ne permet pas confortablement une ACL numérotée.

E1. Créez l'ACL nommée `FILTRE_LAN` reprenant la règle R1, remplacez l'ACL 100 sur l'interface, puis supprimez l'ancienne :

```
R1(config)# ip access-list extended FILTRE_LAN
R1(config-ext-nacl)# deny icmp 192.168.1.0 0.0.0.255 1.1.1.0 0.0.0.3
R1(config-ext-nacl)# permit ip any any
R1(config-ext-nacl)# exit
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip access-group FILTRE_LAN in
R1(config-if)# exit
R1(config)# no access-list 100
```

E2. Avec `show ip interface gigabitEthernet 0/0`, quelle ACL est appliquée en entrée ? Pourquoi l'ACL 100 n'est-elle plus active alors qu'on ne l'a pas retirée de l'interface avec `no ip access-group` ?

`show ip interface gigabitEthernet 0/0` indique « Inbound access list is FILTRE_LAN ». L'ACL 100 n'est plus active parce qu'une interface n'accepte qu'une ACL par sens et par protocole : la commande `ip access-group FILTRE_LAN in` a **remplacé** l'ACL 100.

E3. **Expérience – l'ordre compte.** Ajoutez à la fin de `FILTRE_LAN` une règle qui refuse tout le trafic IP du LAN vers `host 172.16.1.100`. Lancez les tests **6 et 7** puis `show access-lists FILTRE_LAN`. La règle est-elle utilisée ? Pourquoi ? (Regardez les compteurs de correspondances.)

La règle ajoutée (`deny ip 192.168.1.0 0.0.0.255 host 172.16.1.100`, séquence 30) arrive **après** `permit ip any any` (séquence 20). Les tests 6 (ping) et 7 (HTTP) restent OK et son compteur reste à 0 : chaque paquet correspond déjà à `permit ip any any`, et la lecture s'arrête à la première correspondance. La règle n'est jamais atteinte.

E4. **Corrigez.** Écrivez et insérez, à la bonne position, les règles qui implémentent R3 (l'autorisation HTTP **puis** l'interdiction du reste vers ce serveur), en vous servant des numéros de séquence : `<n°> permit …` insère une ligne, `no <n°>` en supprime une. Supprimez la ligne mal placée de E3.

```
R1(config)# ip access-list extended FILTRE_LAN
R1(config-ext-nacl)# 15 permit tcp 192.168.1.0 0.0.0.255 host 172.16.1.100 eq 80
R1(config-ext-nacl)# 16 deny ip 192.168.1.0 0.0.0.255 host 172.16.1.100
R1(config-ext-nacl)# no 30
```

Ordre obtenu : 10 deny icmp → interconnexion, 15 permit HTTP → Serveur-Agence, 16 deny reste → Serveur-Agence, 20 permit ip any any. Ma version recréée (séquences 10, 20, 30, 40) respecte exactement cet ordre : elle est correcte.

> **ATTENTION**
> Si votre version de Packet Tracer refuse les numéros de séquence, supprimez l'ACL (`no ip access-list extended FILTRE_LAN`) puis recréez-la **dans le bon ordre** (règles spécifiques d'abord, règle générale ensuite). Après une modification, l'ACL reste appliquée à l'interface.

E5. Lancez les tests **1 à 10** (non-régression) et complétez la colonne « Obtenu après ACL ». Capturez `show access-lists`.

Résultats reportés dans le tableau de la partie A. `show access-lists` : la règle 30 (deny vers 172.16.1.100) compte 16 correspondances (ping et FTP refusés), la règle 40 compte 4 correspondances.

> **BONNE PRATIQUE**
> Après **chaque** modification d'ACL, rejouez **tous** les tests, pas seulement celui visé : une règle ajoutée peut casser un flux qui fonctionnait.

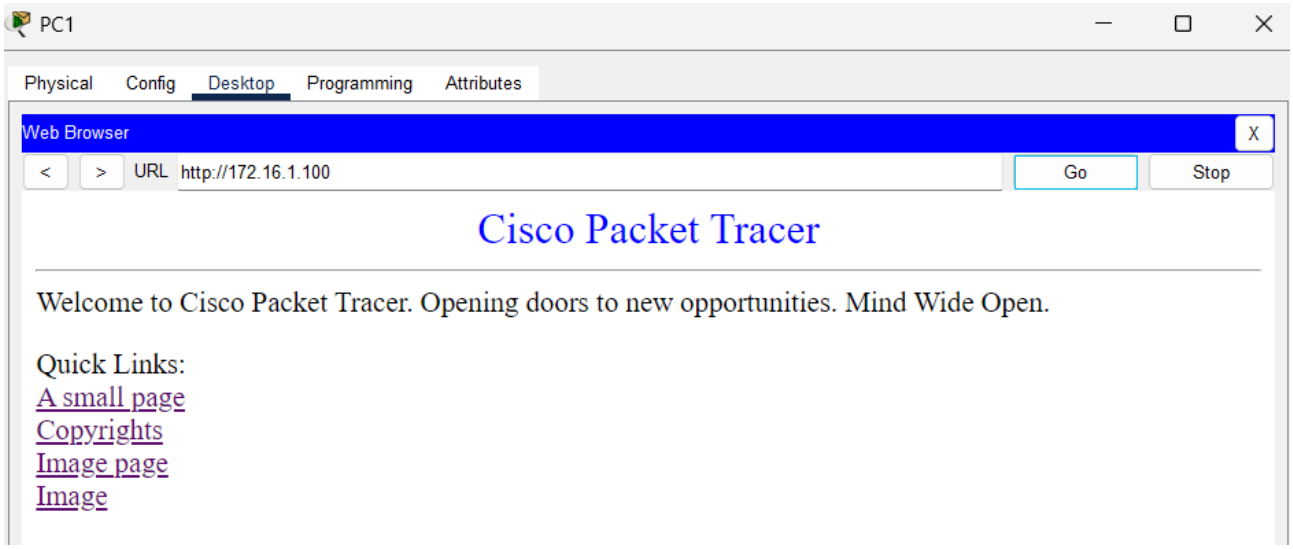
Création de l'ACL nommée FILTRE_LAN (règles R1 et R3 fusionnées)

```
R1#
R1#en
R1#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#no access-list 100
R1(config)#ip access-list extended FILTRE_LAN
R1(config-ext-nacl)#deny icmp 192.168.1.0 0.0.0.255 1.1.1.0 0.0.0.3
R1(config-ext-nacl)#permit tcp 192.168.1.0 0.0.0.255 host 172.16.1.100 eq 80
R1(config-ext-nacl)# eq 80
                    ^
% Invalid input detected at '^' marker.

R1(config-ext-nacl)# eq 80
                    ^
% Invalid input detected at '^' marker.

R1(config-ext-nacl)#deny ip 192.168.1.0 0.0.0.255 host 172.16.1.100
R1(config-ext-nacl)#permit ip any any
R1(config-ext-nacl)#exit
R1(config)#interface gigabitEthernet 0/0
R1(config-if)#ip access-group FILTRE_LAN in
R1(config-if)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console
```

TEST 7 : PC1 → HTTP Serveur-Agence (http://172.16.1.100) = OK



Depuis PC1, ping 172.16.1.100 = KO

```
C:\>ping 172.16.1.100

Pinging 172.16.1.100 with 32 bytes of data:

Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.
Reply from 192.168.1.1: Destination host unreachable.

Ping statistics for 172.16.1.100:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),

C:\>
```

ftp 172.16.1.100 = KO

```
C:\>ftp 172.16.1.100
Trying to connect...172.16.1.100

%Error opening ftp://172.16.1.100/ (Timed out)
.



(Disconnecting from ftp server)
```

Taper show access-lists sur R1

```
R1>en
R1#show access-lists
Extended IP access list FILTRE_LAN
    10 deny icmp 192.168.1.0 0.0.0.255 1.1.1.0 0.0.0.3
    20 permit tcp 192.168.1.0 0.0.0.255 host 172.16.1.100 eq www
    30 deny ip 192.168.1.0 0.0.0.255 host 172.16.1.100 (16 match(es))
    40 permit ip any any (4 match(es))
```

Partie F – Défi : filtrer de l'agence vers le siège, règle R4 (50 min)

> **OBJECTIF**
> Ici, **vous concevez** : on passe d'une logique « tout est permis sauf… » à une logique **liste blanche** (« tout est interdit sauf… »).

F1. **Sur papier d'abord.** Complétez le tableau de conception de l'ACL `AGENCE_IN` (une ligne par règle, dans l'ordre). Indices : le HTTP est du **TCP/80**, le DNS du **UDP/53** ; les protocoles TCP et ICMP disposent de mots-clés pour reconnaître des **réponses** (`established`, `echo-reply` – voir annexe).

Rang	Action	Protocole	Source	Destination	Port / option	Rôle
1	permit	tcp	172.16.1.0 0.0.0.255	host 192.168.1.251	eq 80	HTTP vers le serveur Web
2	permit	udp	172.16.1.0 0.0.0.255	host 192.168.1.252	eq 53	Requêtes DNS
3	permit	tcp	172.16.1.0 0.0.0.255	192.168.1.0 0.0.0.255	established	Réponses TCP aux connexions du siège
4	permit	icmp	172.16.1.0 0.0.0.255	192.168.1.0 0.0.0.255	echo-reply	Réponses aux pings du siège
5	deny	ip	172.16.1.0 0.0.0.255	192.168.1.0 0.0.0.255	–	Tout le reste agence → siège
6	permit	ip	any	any	–	Autres trafics (hors LAN siège)

F2. Sur quel routeur, quelle interface et quel sens l'appliquer ? Justifiez.

Routeur **R2**, interface **Gi0/0**, sens **in**. C'est une ACL étendue, qu'on place au plus près de la source (le LAN de l'agence) : le trafic de l'agence entre dans R2 par Gi0/0. Il n'y a pas de conflit avec l'ACL 10, appliquée sur la même interface mais en sens out.

F3. **Expérience.** Créez d'abord une version **naïve** qui contient uniquement : autorisation HTTP vers le serveur Web, autorisation DNS vers le serveur DNS, interdiction du reste de l'agence vers le LAN, puis `permit ip any any`. Lancez les tests **7 et 9**. Que constatez-vous ? Expliquez précisément le trajet du paquet qui est bloqué.

Avec la version naïve, les tests 7 (PC1 → HTTP Serveur-Agence) et 9 (Laptop1 → HTTP Serveur-Agence) deviennent KO.

Trajet : la requête HTTP part du siège, est autorisée par FILTRE_LAN sur R1 et par l'ACL 10 sur R2, puis arrive au Serveur-Agence. Celui-ci répond (TCP, port source 80, vers 192.168.1.x). Cette **réponse** entre dans R2 par Gi0/0 et passe dans AGENCE_IN : elle ne correspond ni à la règle HTTP vers .251 ni à la règle DNS, mais à `deny ip 172.16.1.0 0.0.0.255 192.168.1.0 0.0.0.255`. Elle est rejetée par R2. C'est la conséquence directe du filtrage sans état.

F4. **Complétez** l'ACL pour que les réponses aux connexions initiées depuis le siège passent à nouveau (tests 4, 7 et 9), sans ouvrir l'accès de l'agence au reste du LAN.

```
R2(config)# ip access-list extended AGENCE_IN
R2(config-ext-nacl)# permit tcp 172.16.1.0 0.0.0.255 host 192.168.1.251 eq 80
R2(config-ext-nacl)# permit udp 172.16.1.0 0.0.0.255 host 192.168.1.252 eq 53
R2(config-ext-nacl)# permit tcp 172.16.1.0 0.0.0.255 192.168.1.0 0.0.0.255 established
```

```
R2(config-ext-nacl)# permit icmp 172.16.1.0 0.0.0.255 192.168.1.0 0.0.0.255 echo-reply
R2(config-ext-nacl)# deny ip 172.16.1.0 0.0.0.255 192.168.1.0 0.0.0.255
R2(config-ext-nacl)# permit ip any any
R2(config)# interface gigabitEthernet 0/0
R2(config-if)# ip access-group AGENCE_IN in
```

Les règles `established` et `echo-reply`, placées avant le deny, laissent passer les réponses (tests 4, 7 et 9 OK) sans ouvrir l'accès de l'agence au reste du LAN. Ma configuration (capture ci-dessous) est correcte.

F5. Rejouez les **14 tests** et complétez le tableau de la partie A. Chaque écart avec vos prévisions doit être expliqué.

Tableau de la partie A complété. Aucun écart avec les prévisions sur les tests capturés.

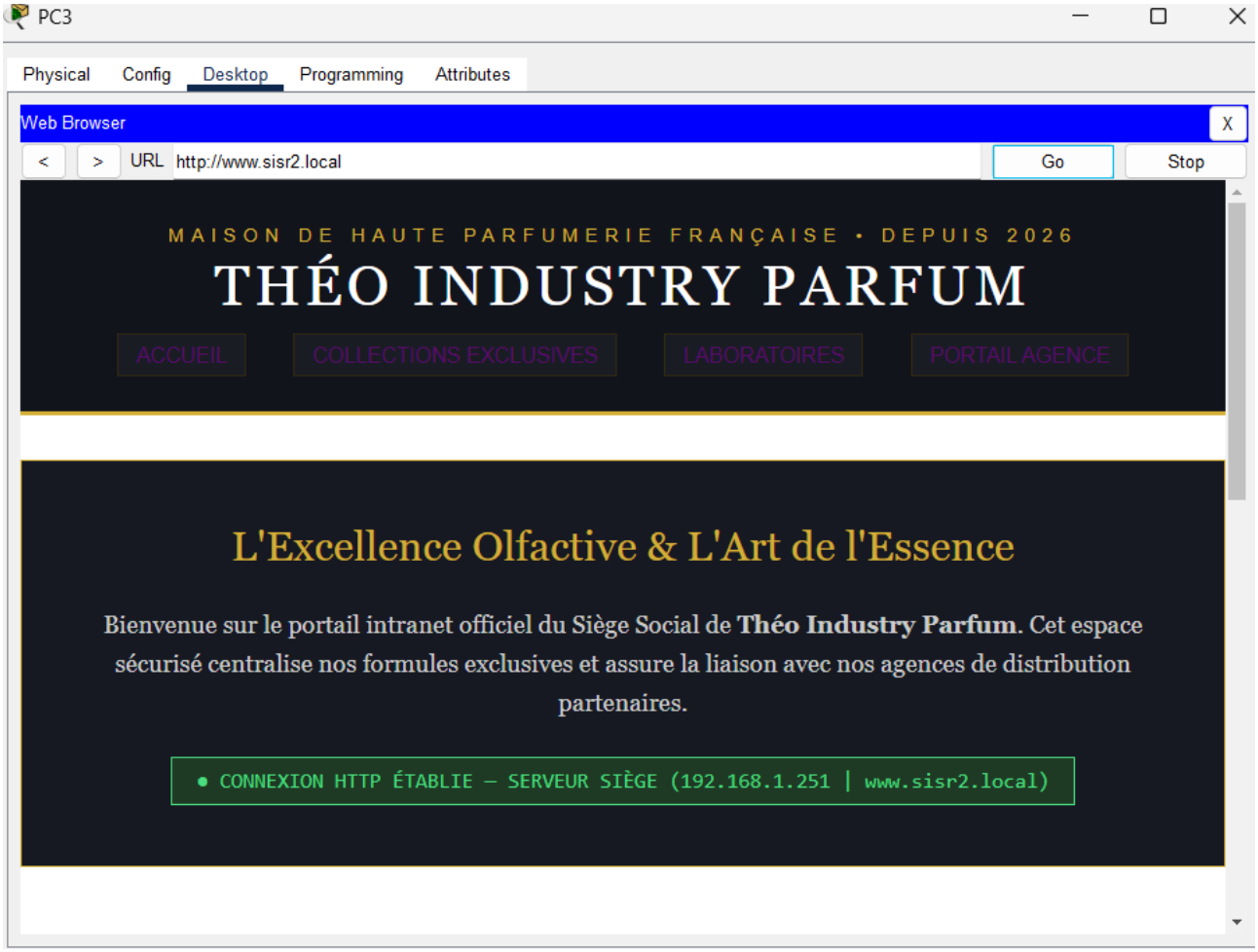
> **LIMITE À CONNAÎTRE**
> Le mot-clé `established` se contente de reconnaître des segments TCP portant le drapeau ACK ou RST : un attaquant peut en fabriquer. Il n'existe pas d'équivalent pour UDP. Pour un vrai suivi de connexions, il faut un **pare-feu à états** (voir bilan).

Création et application de l'ACL nommée AGENCE_IN sur R2 (Gi0/0, in)

```
R2>
R2>en
R2#conf t
Enter configuration commands, one per line.  End with CNTL/Z.
R2(config)#ip access-list extended AGENCE_IN
R2(config-ext-nacl)#permit tcp 172.16.1.0 0.0.0.255 host 192.168.1.251 eq 80
R2(config-ext-nacl)#permit udp 172.16.1.0 0.0.0.255 host 192.168.1.252 eq 53
R2(config-ext-nacl)#permit tcp 172.16.1.0 0.0.0.255 192.168.1.0 0.0.0.255 established
R2(config-ext-nacl)#permit icmp 172.16.1.0 0.0.0.255 192.168.1.0 0.0.0.255 echo-reply
R2(config-ext-nacl)#deny ip 172.16.1.0 0.0.0.255 192.168.1.0 0.0.0.255
R2(config-ext-nacl)#permit ip any any
R2(config-ext-nacl)#exit
R2(config)#interface gigabitEthernet 0/0
R2(config-if)#ip access-group AGENCE_IN in
R2(config-if)#end
R2#
%SYS-5-CONFIG_I: Configured from console by console

R2#
R2#
```

TEST 11 : PC3 → http://www.sisr2.local = OK (résolution DNS + HTTP)



PC3 ping PC1 = KO

```
C:\>ping 192.168.1.111

Pinging 192.168.1.111 with 32 bytes of data:

Reply from 172.16.1.1: Destination host unreachable.
Reply from 172.16.1.1: Destination host unreachable.
Reply from 172.16.1.1: Destination host unreachable.
Reply from 172.16.1.1: Destination host unreachable.

Ping statistics for 192.168.1.111:
    Packets: Sent = 4, Received = 0, Lost = 4 (100% loss),
```

PC1 ping PC3 = OK

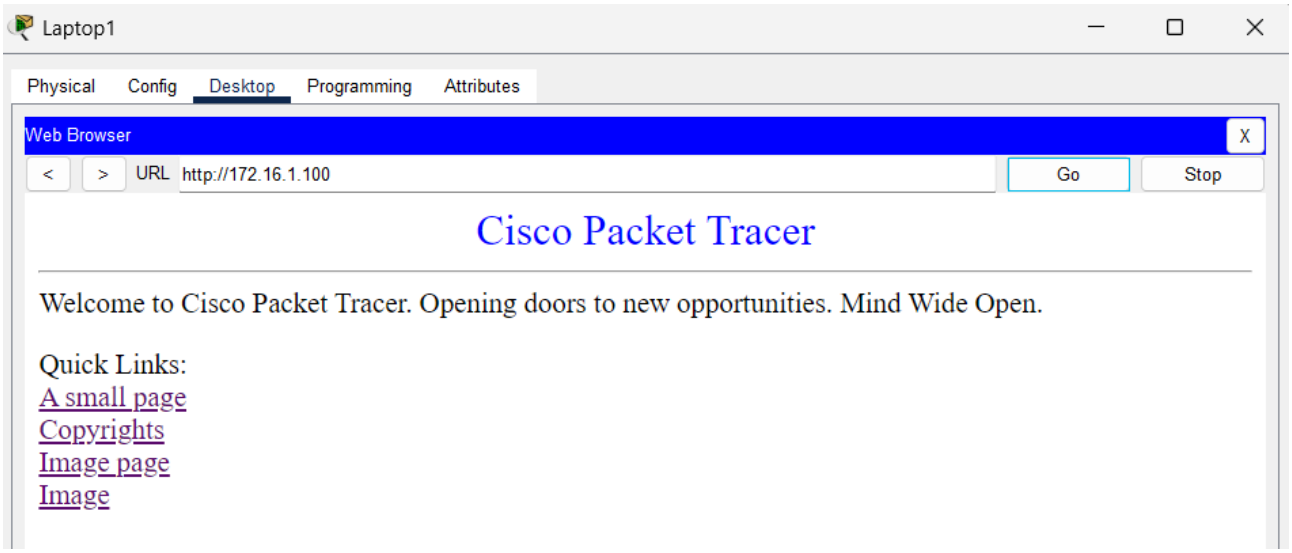
```
C:\>ping 172.16.1.10

Pinging 172.16.1.10 with 32 bytes of data:

Reply from 172.16.1.10: bytes=32 time=19ms TTL=126
Reply from 172.16.1.10: bytes=32 time=9ms TTL=126
Reply from 172.16.1.10: bytes=32 time=13ms TTL=126
Reply from 172.16.1.10: bytes=32 time=13ms TTL=126

Ping statistics for 172.16.1.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 9ms, Maximum = 19ms, Average = 13ms
```

TEST 9 : Laptop1 → http://172.16.1.100 = OK



Partie G – Dépannage : ACL en panne (20 min)

Un collègue a écrit les ACL suivantes. Pour chaque cas, **identifiez l'erreur, expliquez le symptôme et proposez la correction** (testez-la dans Packet Tracer si besoin).

> **MÉTHODE DE DIAGNOSTIC**
> 1) `show access-lists` : les règles existent-elles, dans le bon ordre, avec des compteurs qui évoluent ? 2) `show ip interface <int>` : l'ACL est-elle appliquée, sur la bonne interface, dans le bon sens ? 3) Mode **Simulation** : à quel équipement le paquet est-il rejeté ? 4) Relire chaque wildcard.

Cas 1 – « Le FTP est toujours possible »

```
R1(config)# access-list 101 permit ip any any
R1(config)# access-list 101 deny tcp 192.168.1.0 0.0.0.255 host 172.16.1.100 eq 21
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip access-group 101 in
```

Erreur : `permit ip any any` est placée en première position. **Symptôme :** tous les paquets correspondent dès la première règle, la règle `deny … eq 21` n'est jamais lue et le FTP passe.

`no access-list 101`, puis recréer l'ACL avec le deny en premier : `access-list 101 deny tcp 192.168.1.0 0.0.0.255 host 172.16.1.100 eq 21`, puis `access-list 101 permit ip any any`.

Cas 2 – « Depuis que j'ai posé l'ACL, le LAN n'accède plus à rien en dehors de lui-même »

```
R1(config)# access-list 102 deny icmp 192.168.1.0 0.0.0.255 1.1.1.0 0.0.0.3
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip access-group 102 in
```

Erreur : il manque `permit ip any any` à la fin. **Symptôme :** le deny implicite refuse tout le trafic du LAN qui entre dans R1. Seuls les échanges internes au LAN, qui ne passent pas par le routeur, fonctionnent encore.

ajouter `access-list 102 permit ip any any`.

Cas 3 – « La règle n'a aucun effet sur le trafic HTTP de PC1 »

```
R1(config)# access-list 103 deny tcp 192.168.1.0 255.255.255.0 host 172.16.1.100 eq 80
R1(config)# access-list 103 permit ip any any
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip access-group 103 in
```

Erreur : le masque de sous-réseau `255.255.255.0` a été écrit à la place du wildcard `0.0.0.255`. **Symptôme :** avec ce « wildcard », seuls les bits du dernier octet sont comparés (ils doivent valoir 0) : la règle ne vise que les adresses en x.x.x.0. PC1 (192.168.1.111) n'y correspond pas, donc son HTTP passe par `permit ip any any`.

recréer l'ACL avec `access-list 103 deny tcp 192.168.1.0 0.0.0.255 host 172.16.1.100 eq 80` puis `access-list 103 permit ip any any`.

Cas 4 – « L'ACL est bonne mais les compteurs restent à 0 »

```
R1(config)# access-list 104 deny icmp 192.168.1.0 0.0.0.255 1.1.1.0 0.0.0.3
R1(config)# access-list 104 permit ip any any
R1(config)# interface gigabitEthernet 0/0
R1(config-if)# ip access-group 104 out
```

Erreur : l'ACL est appliquée en **out** sur Gi0/0. **Symptôme :** le trafic du LAN vers 1.1.1.0/30 **entre** par Gi0/0 et sort par l'interface série. En sortie de Gi0/0, l'ACL ne voit que le trafic qui va vers le LAN, dont la source n'est jamais 192.168.1.0/24 : la règle deny ne correspond jamais et les compteurs restent à 0.

sur Gi0/0, `no ip access-group 104 out` puis `ip access-group 104 in`.

Partie H – Bilan, livrables et pour aller plus loin

Questions de bilan

H1. Les ACL Cisco sont dites « sans état » (stateless). Illustrez avec un exemple vu dans ce TP. Quelle solution technique répond à cette limite ?

Exemple : en D4 (b), le ping de R2 vers PC1 échoue. La demande passe, mais la réponse de PC1 est rejetée par l'ACL de R1, qui ne sait pas qu'il s'agit d'une réponse. Même chose en F3 : les réponses HTTP du Serveur-Agence sont bloquées par AGENCE_IN.

Solution : un **pare-feu à états** (stateful : pfSense, Cisco ASA, IOS Zone-Based Firewall…). Il mémorise les connexions ouvertes depuis l'intérieur et autorise automatiquement leur trafic retour.

H2. Pourquoi filtrer sur l'adresse IP d'un poste est-il fragile (DHCP, usurpation d'adresse) ? Citez deux pistes pour fiabiliser.

Une adresse IP n'identifie pas un poste de façon fiable : l'utilisateur peut la changer (vu en C6), elle varie avec le DHCP et elle peut être usurpée par un autre poste.

Pistes pour fiabiliser : (1) segmenter en VLAN et filtrer par sous-réseau plutôt que par poste ; (2) lier l'adresse au poste (réservation DHCP, port-security sur le switch, DHCP snooping et IP Source Guard) ; (3) authentifier les postes avec 802.1X.

H3. Quand faut-il terminer une ACL par `permit ip any any`, et quand faut-il au contraire s'en abstenir ? Quel est le risque de chaque choix ?

Avec `permit ip any any` à la fin (liste noire) : on bloque quelques flux précis et tout le reste passe, comme pour FILTRE_LAN. C'est adapté quand on veut seulement interdire certains flux. Risque : tout flux non prévu est autorisé, ce qui peut ouvrir un accès involontaire.

Sans (liste blanche) : seul ce qui est explicitement autorisé passe, comme l'accès de l'agence au siège dans AGENCE_IN. C'est adapté aux zones sensibles. Risque : oublier un flux indispensable (DNS, DHCP, réponses…) et casser un service.

H4. Où place-t-on une ACL standard ? une ACL étendue ? Pourquoi cette différence ?

ACL standard : **au plus près de la destination**. Elle ne voit que la source : placée près de la source, elle bloquerait cette source vers toutes les destinations.

ACL étendue : **au plus près de la source**. Elle est assez précise pour ne viser que le flux indésirable : on l'arrête le plus tôt possible, sans le faire traverser le réseau pour rien.

H5. Comment vérifier rapidement qu'une ACL est bien appliquée, et où s'arrêtent les paquets qu'elle refuse ?

`show ip interface <int>` indique l'ACL appliquée en entrée (Inbound) et en sortie (Outgoing). `show access-lists` permet de vérifier l'ordre des règles et les compteurs (faire `clear access-list counters`, lancer un test, puis regarder quelle règle s'incrémente). `show running-config` contrôle les lignes access-list et ip access-group.

Pour savoir où le paquet est arrêté : le mode Simulation de Packet Tracer montre l'équipement qui le rejette, et le message « Destination host unreachable » indique le routeur qui filtre (192.168.1.1 = R1, 1.1.1.1 ou 172.16.1.1 = R2).

Livrables attendus

- **Compte-rendu Word** : topologie, plan d'adressage, tableau des 14 tests complété, et pour chaque partie les commandes, les captures (`show access-lists` avec compteurs, tests) et les réponses aux questions.
- **Fichier Packet Tracer** final (`TP02_final.pkt`).
- **Sortie de `show running-config`** de R1 et R2 (au minimum les lignes `access-list`, `ip access-list` et `ip access-group`).
- Une **synthèse d'une demi-page** utilisable dans votre portfolio : problème posé, solution, difficultés rencontrées.

Pour aller plus loin (bonus)

- **Sécuriser l'accès distant au routeur** : configurer Telnet/SSH sur R1 (`line vty 0 4`) et n'autoriser que PC1 grâce à une ACL standard appliquée avec `access-class 20 in` sur les lignes VTY.
- **ACL et VLANs** : découpez le LAN du siège en VLAN 10 (Administration), VLAN 20 (Informatique) et VLAN 30 (Marketing) avec routage inter-VLAN (voir votre fiche « Configuration des VLANs sur un switch »), puis écrivez les ACL pour que le VLAN 30 ne puisse pas joindre le VLAN 20 alors que le VLAN 10 joint tout. Où les placer ?

- **Journalisation** : ajoutez le mot-clé `log` à une règle `deny` et observez la sortie (`show logging`).

SAUVEGARDE R1 configuration :

```
R1#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
R1#
```

SAUVEGARDE R2 configuration :

```
R2>en
R2#copy running-config startup-config
Destination filename [startup-config]?
Building configuration...
[OK]
R2#
```

R1 : show running-config (extrait : interfaces et ACL)

```
interface GigabitEthernet0/0
 ip address 192.168.1.1 255.255.255.0
 ip access-group FILTRE_LAN in
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
 no ip address
 duplex auto
 speed auto
 shutdown
!
interface GigabitEthernet0/2
 no ip address
 duplex auto
 speed auto
 shutdown
!
interface Serial0/3/0
 ip address 1.1.1.2 255.255.255.252
 clock rate 64000
!
interface Serial0/3/1
 no ip address
 clock rate 2000000
 shutdown
!
interface Vlan1
 no ip address
 shutdown
!
ip classless
ip route 172.16.1.0 255.255.255.0 1.1.1.1
!
ip flow-export version 9
!
!
ip access-list extended FILTRE_LAN
 deny icmp 192.168.1.0 0.0.0.255 1.1.1.0 0.0.0.3
 permit tcp 192.168.1.0 0.0.0.255 host 172.16.1.100 eq www
 deny ip 192.168.1.0 0.0.0.255 host 172.16.1.100
 permit ip any any
!
!
```

R2 : show running-config (extrait : interfaces et ACL)

```
interface GigabitEthernet0/0
 ip address 172.16.1.1 255.255.255.0
 ip access-group AGENCE_IN in
 ip access-group 10 out
 duplex auto
 speed auto
!
interface GigabitEthernet0/1
 no ip address
 duplex auto
 speed auto
 shutdown
!
interface GigabitEthernet0/2
 no ip address
 duplex auto
 speed auto
 shutdown
!
interface Serial0/3/0
 ip address 1.1.1.1 255.255.255.252
!
interface Serial0/3/1
 no ip address
 clock rate 2000000
 shutdown
!
interface Vlan1
 no ip address
 shutdown
!
ip classless
ip route 192.168.1.0 255.255.255.0 1.1.1.2
!
ip flow-export version 9
!
!
access-list 10 deny host 192.168.1.112
access-list 10 permit any
ip access-list extended AGENCE_IN
 permit tcp 172.16.1.0 0.0.0.255 host 192.168.1.251 eq www
 permit udp 172.16.1.0 0.0.0.255 host 192.168.1.252 eq domain
 permit tcp 172.16.1.0 0.0.0.255 192.168.1.0 0.0.0.255 established
 permit icmp 172.16.1.0 0.0.0.255 192.168.1.0 0.0.0.255 echo-reply
 deny ip 172.16.1.0 0.0.0.255 192.168.1.0 0.0.0.255
 permit ip any any
!
!
!
!
!
line con 0
```

Synthèse pour le portfolio

Problème posé. L'entreprise SISR2 relie son siège (192.168.1.0/24 : serveurs DHCP, DNS et Web) à une agence (172.16.1.0/24) par une liaison série. Par défaut, les deux routeurs transmettent tout le trafic. Suite à un audit, le RSSI demande d'appliquer une politique de filtrage en quatre règles : interdire le ping vers l'interconnexion, couper PC2 de l'agence, limiter l'accès au Serveur-Agence au seul HTTP et n'ouvrir que le Web et le DNS du siège à l'agence.

Solution mise en œuvre. Sous Packet Tracer, j'ai d'abord monté et routé la maquette (routes statiques), puis réalisé 14 tests de référence. J'ai ensuite traduit chaque règle en ACL Cisco : une ACL standard (ACL 10) en sortie de R2 pour PC2 ; une ACL étendue nommée FILTRE_LAN en entrée de R1, qui fusionne le blocage ICMP et l'accès HTTP seul au Serveur-Agence ; une ACL étendue AGENCE_IN en entrée de R2, en logique liste blanche, avec les mots-clés established et echo-reply pour laisser passer les réponses. Chaque ACL a été validée par les tests autorisés et interdits et par les compteurs de show access-lists.

Difficultés rencontrées. L'interface série du module HWIC-2T était Se0/3/0 et non Se0/0/0 comme dans l'énoncé. L'ordre des règles est déterminant : une règle générale placée trop tôt rend les suivantes inutiles. Une interface n'accepte qu'une ACL par sens, d'où la fusion des règles dans FILTRE_LAN. Enfin, les ACL étant sans état, les réponses doivent être autorisées explicitement.

Ce que j'en retiens. Les ACL sont une première brique de sécurité simple et efficace, mais limitée : pas de suivi des connexions et un filtrage fragile par adresse IP. En production, je les compléterais par une segmentation en VLAN, un pare-feu à états et l'authentification des postes.

Annexe – Aide-mémoire ACL

1. Les règles d'or

1. Le routeur lit l'ACL **de haut en bas** ; **la première règle qui correspond s'applique**, les suivantes sont ignorées.
2. Il existe un **`deny` implicite** à la fin de toute ACL : ce qui n'est pas explicitement autorisé est refusé.
3. **Une ACL par interface, par sens (in/out) et par protocole** (IPv4). Appliquer une nouvelle ACL au même sens remplace la précédente.
4. Une ACL **ne filtre pas le trafic généré par le routeur** lui-même.
5. Une ACL classique est **sans état** : la réponse à une requête autorisée doit être autorisée explicitement.
6. Appliquer à une interface une ACL **inexistante ou vide** laisse **tout passer**.

2. ACL standard et ACL étendue

	ACL standard	ACL étendue
Numéros	1 à 99 et 1300 à 1999	100 à 199 et 2000 à 2699
Critères	Adresse **source** uniquement	Protocole, source, destination, ports, options
Placement	Au plus **près de la destination**	Au plus **près de la source**
Pourquoi ?	Ne connaît pas la destination : placée près de la source, elle bloquerait tout, partout	Très précise : on évite de faire voyager inutilement un trafic déjà condamné

3. Wildcard mask

Wildcard = 255.255.255.255 – masque de sous-réseau. `0` : le bit doit correspondre ; `1` : le bit est ignoré.

Préfixe	Masque	Wildcard	Adresses
/8	255.0.0.0	0.255.255.255	16 777 216
/16	255.255.0.0	0.0.255.255	65 536
/24	255.255.255.0	0.0.0.255	256
/25	255.255.255.128	0.0.0.127	128
/26	255.255.255.192	0.0.0.63	64
/28	255.255.255.240	0.0.0.15	16
/30	255.255.255.252	0.0.0.3	4

Raccourcis : `host 10.0.0.1` ≡ `10.0.0.1 0.0.0.0` (une seule adresse) – `any` ≡ `0.0.0.0 255.255.255.255` (toutes les adresses).

4. Syntaxe

Standard :

```
access-list <1-99> {permit | deny} {host <IP> | <réseau> <wildcard> | any}
```

Étendue :

```
access-list <100-199> {permit|deny} <protocole> <source> <wildcard-src> [port-src]
                      <destination> <wildcard-dst> [port-dst] [option]
```

Élément	Valeurs possibles
Protocole	`ip` (tout IP), `tcp`, `udp`, `icmp`
Opérateurs de port (TCP/UDP)	`eq` (égal), `neq` (différent), `lt` (inférieur), `gt` (supérieur), `range <min> <max>`
Ports courants	FTP `21` (`ftp`) – SSH `22` – Telnet `23` – SMTP `25` – DNS `53` (UDP) – HTTP `80` (`www`) – HTTPS `443`
Options utiles	TCP : `established` (segment avec ACK/RST = réponse) – ICMP : `echo` (demande de ping), `echo-reply` (réponse) – `log`

Application sur une interface :

```
Router(config)# interface <interface>
Router(config-if)# ip access-group <numéro | nom> {in | out}
```

5. ACL nommées

```
Router(config)# ip access-list {standard | extended} <NOM>
Router(config-ext-nacl)# [<n° séquence>] {permit | deny} <règle>
Router(config-ext-nacl)# no <n° séquence>          ! supprime une ligne
```

Avantages : nom explicite, insertion/suppression d'une ligne sans tout recréer. Bonne pratique : documenter avec `remark <commentaire>`.

6. Vérification et gestion

Commande	Rôle
`show access-lists`	Affiche toutes les ACL avec leurs numéros de séquence et **compteurs de correspondances**
`show ip interface <int>`	Indique l'ACL appliquée en entrée (Inbound) et en sortie (Outgoing)
`show running-config`	Vérifie les lignes `access-list` et `ip access-group`
`clear access-list counters`	Remet les compteurs à zéro avant un test
`no access-list <n°>`	Supprime **toute** l'ACL numérotée
`no ip access-list extended <nom>`	Supprime une ACL nommée
`no ip access-group <ACL> {in|out}`	(mode interface) Retire l'ACL de l'interface sans la supprimer

7. Erreurs fréquentes

- Règle générale placée **avant** une règle spécifique (jamais atteinte).
- **Masque** de sous-réseau écrit à la place du **wildcard** (`255.255.255.0` au lieu de `0.0.0.255`).
- ACL appliquée au **mauvais sens** ou sur la **mauvaise interface** (compteurs à 0).
- Oubli du **`permit` final** : le `deny` implicite bloque tout le reste.
- **Ordre des paramètres** : source avant destination ; port de destination après l'adresse de destination.
- Oubli du **retour** d'un flux autorisé (ACL sans état).
- Blocage de sa **propre administration** à distance : testez toujours depuis la console avant de filtrer Telnet/SSH.